Tan *et al. Cybersecurity* (2026) 9:180
https://doi.org/10.1186/s42400-026-00616-0

Cybersecurity

RESEARCH Open Access

TLBAC: a zero-trust based cloud-edge-endpoint access control system using trusted labels

Ru Tan[1,2], Yingjun Zhang[3], Wei Liu[1], Baihang Liu[1,2], Yaqin Cao[1,2*], Xinyu Liu[1], Yuling Liu[1,2] and Qixu Liu[1,2*]

Abstract

With the rapid development of cloud computing, enterprises have increasingly migrated their servers and services from internal networks to cloud environments. Traditional boundary-based protection mechanisms are no longer sufficient for addressing the security requirements of modern cloud-based systems. As cyberattacks continue to evolve, ensuring the long-term, secure, and reliable operation of increasingly complex IT systems has become a significant challenge. Consequently, researchers have proposed various fine-grained access control approaches. Fine-grained access control remains a significant challenge in complex cloud-edge-endpoint collaboration scenarios. Existing approaches often rely on intricate policy definitions to achieve granular control, which can lead to increased computational overhead and degraded system performance. In this paper, we propose a lightweight, Zero-Trust-based Cloud-Edge-Endpoint Access Control System—TLBAC. By embedding trusted labels into TCP packets at the security endpoint (Endpoint), TLBAC enables traffic blocking and forwarding through access control policies issued by the Security Cloud Brain (Cloud) via the Edge Decision Gateway (Edge). This framework achieves fine-grained access control through trusted labels in a cloud-edge-endpoint architecture. To evaluate the effectiveness of TLBAC, four experiments are designed. The first experiment examines the impact of trusted labels insertion on TCP traffic packet transmission behavior. The experimental results show that even under high-latency, significant jitter, and high packet loss conditions in an LTE network, TCP packets with embedded trusted labels maintain stable and reliable data transmission. The second experiment assesses the resource consumption of TLBAC, focusing on CPU and memory overhead. The experimental results indicate that TLBAC's resource consumption rate is only ± 0.2%. The third experiment simulates a realistic attack environment by launching attacks against the protected system from an adversarial perspective to validate TLBAC's defensive capabilities. The experimental results demonstrate that TLBAC successfully detected and blocked all attacks in over thirty vulnerability cases involving different components, versions, and exploitation methods. The fourth experiment evaluates the practical operational capability of TLBAC in a multi-tenant environment. The results show that TLBAC can achieve connection-oriented fine-grained access control with low latency, while maintaining policy accuracy and isolation in multi-tenant scenarios.

Keywords Cloud-edge-endpoint, Trusted labels, Fine-grained access control, Zero-trust

*Correspondence:
Yaqin Cao
caoyaqin@iie.ac.cn
Qixu Liu
liuqixu@iie.ac.cn
Full list of author information is available at the end of the article

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Introduction

As cloud computing becomes increasingly central to enterprise IT architectures and its application scenarios expand, enterprise computing environments are evolving from closed, centralized networks into highly distributed and cross-domain ecosystems (Wang and Du 2022). Accordingly, the security boundaries of cloud environments differ fundamentally from those of traditional network architectures. (1) Cloud computing (Deng et al. 2024) disrupts the intranet-centric security assumption by decoupling security boundaries from physical networks. Traditional network security models rely on firewalls and demilitarized zones to separate trusted internal networks from untrusted external networks. With the widespread adoption of cloud computing, however, cloud infrastructure can no longer be enclosed within conventional firewall-defined perimeters, rendering security boundaries independent of fixed network locations or physical facilities (Pramod and Singh 2023). (2) Diverse application scenarios drive the evolution of security boundaries from a single perimeter to distributed and multidimensional boundaries (Moubayed et al. 2019). The integration of cloud computing with mobile internet (Mao et al. 2017) , the Internet of Things (Atzori et al. 2010), and big data (Oussous et al. 2018) technologies expands enterprise application scenarios and business boundaries, resulting in multiple concurrent security boundaries across cloud-based and on-premises environments, endpoint devices, and data exchange channels (Liu et al. 2024). (3) Boundary control shifts from network connectivity to identity, context, and business trust (Ali et al. 2024). In cloud and cloud-native environments, resources are highly decoupled and services interact primarily through APIs and microservices (Di Francesco et al. 2019), shifting security risks from the network layer to the misuse of identities (Aldea and Bocu 2025), privileges (Yang et al. 2025), and data access (Xu et al. 2018). This shift has further driven the adoption of security paradigms such as Zero Trust (Gambo and Almulhem 2025), which emphasize continuous verification rather than implicit trust in network locations.

With the widespread adoption of cloud computing, its elasticity, virtualization, distribution, and service-oriented characteristics have enhanced resource efficiency and business agility, while also introducing new security challenges. (1) Unlike traditional IT environments with relatively stable network topologies, cloud environments support the on-demand creation and destruction of instances, containers, and services, with IP addresses and resource mappings frequently changing (dos Santos et al. 2016; Pauley et al. 2022). Such highly dynamic resource characteristics make it difficult for static rule–based access control mechanisms to remain effective over time. (2) Diversified access subjects and paths disrupt the traditional "human–network–application" model (Li et al. 2021). Access is no longer confined to a single network channel but spans interactions among cloud services (Diekmann et al. 2019), across cloud and on-premises environments (Lin et al. 2024), and across organizational and trust boundaries (Ahmed et al. 2019), which traditional user- or network-centric access control models cannot fully capture. (3) The decoupling of data and privileges complicates enforcement of the principle of least privilege. Conventional access control mechanisms often lack fine-grained (Yu et al. 2010), context-aware authorization capabilities (Oluwatimi et al. 2018), leading to excessive and long-lived permissions that conflict with cloud security principles such as least privilege, on-demand authorization, and short-lived credentials (Ferretti et al. 2021).

To address the highly distributed and dynamically evolving environments of cloud computing, modern cybersecurity systems increasingly adopt security architectures based on Zero Trust principles, which fundamentally differ from traditional models relying on fixed perimeters and implicit trust. (1) Zero Trust assumes that no network location is inherently trusted. Unlike traditional models that implicitly trust internal networks, Zero Trust requires continuous verification of every access request based on attributes such as identity, device posture, and contextual information. (2) Security control shifts from perimeter-centric protection to access-centric, fine-grained enforcement (Sänger and Abeck 2024). This approach enforces security at the application, service, and data layers, enabling granular access policies and consistent application of the principle of least privilege. (3) Static, long-lived authorizations are replaced by context-aware, dynamic decision-making. Access decisions are made dynamically using real-time contextual information, allowing security policies to adapt to rapidly changing resources and access relationships in cloud environments.

Although Zero Trust architectures effectively address the limitations of traditional perimeter-based security in cloud and distributed environments at the conceptual level, existing Zero Trust implementations still exhibit several unresolved limitations in practical deployment. (1) Limited context awareness and trust evaluation capabilities hinder accurate modeling of dynamic risks. (2) High architectural complexity and deployment costs constrain cross-domain collaboration. (3) Continuous verification mechanisms introduce performance overheads and trade-offs with user experience. To address the aforementioned challenges, this paper proposes an innovative cloud–edge–endpoint collaborative security solution that integrates trusted labels with Zero Trust principles and is implemented at the foundational layer of network protocols. By embedding trusted labels into network

communications, the proposed approach enables lightweight and scalable fine-grained access control. The main contributions of this work are as follows:

- Trusted labels derived from user identity are embedded into TCP traffic packets to enable fine-grained access control.
- We designed a Trusted Labels-Based Access Control (TLBAC) system that realizes policy-driven, systematic access control by organizing the architecture into three functional layers: the Security Cloud Brain (cloud layer), the Edge Detection Gateway (edge layer), and the Security Endpoint (endpoint layer).
- Experiments conducted under real LTE network conditions demonstrate that TLBAC does not compromise the inherent reliability mechanisms of the underlying protocol, while incurring low CPU utilization and memory overhead.
- Through evaluations conducted in multiple vulnerability-enabled environments, we verify that TLBAC can effectively detect and block malicious attacks under realistic threat conditions, providing reliable defensive capabilities.
- Through multi-tenant experiments, we verified that TLBAC can achieve efficient, stable, and well-isolated fine-grained access control under multi-user concurrent scenarios, demonstrating strong security and practical applicability.

Background

Research on cloud-oriented access control technologies

Access control is a fundamental mechanism for ensuring the security and trustworthiness of information systems by regulating access to resources through identity authentication, authorization, and behavioral constraints, thereby safeguarding confidentiality, integrity, and availability. However, as cloud computing reshapes information infrastructures toward highly distributed, elastic, and multi-tenant environments, traditional access control models such as Discretionary Access Control (Lee et al. 2021), Mandatory Access Control (Rossi et al. 2021), and Role-Based Access Control (Zahoor et al. 2023) increasingly struggle to accommodate the dynamic nature, fine-grained requirements, and cross-domain interactions inherent to cloud platforms. In response, recent research on cloud-oriented access control has shifted toward more flexible and expressive paradigms, including Attribute-Based Access Control (Shang et al. 2024), context- and risk-aware dynamic authorization, and fine-grained policy enforcement under the Zero Trust model. These approaches emphasize identity attributes, environmental context, behavioral patterns, and real-time risk assessment as the basis for access decisions, supported by continuous monitoring and verification, thereby enabling adaptive and scalable access control suitable for complex, distributed cloud environments. The traditional access control framework is illustrated in Fig. 1.

Recent studies have explored cloud-oriented access control from complementary perspectives. Qi et al. focus on cryptographically enforced access control under untrusted or semi-trusted cloud assumptions, achieving stronger security guarantees for realistic data-sharing scenarios at the cost of additional overhead in large-scale, dynamic multi-tenant environments (Qi and Zheng 2021). Xue et al. improve scalability and efficiency through policy reuse mechanisms, yet their approach requires further integration with cloud-native runtime infrastructures such as API gateways and service meshes for end-to-end enforcement (Xue et al. 2022). Yang et al. systematically reveal permission proliferation in practical the role-based access control deployments, motivating least-privilege enforcement and permission auditing, while leaving the automatic convergence toward minimal privilege and continuous role evolution as open challenges(Yang et al. 2023). Raffa et al. address the combined risks of cloud permissions, event-driven execution, and application logic—an area largely uncovered by traditional perimeter-based models—although a complete runtime enforcement loop comparable to mandatory access control remains to be established(Raffa et al. 2025).

Zero trust–driven access control for cloud environments

The concept of Zero Trust was first proposed by John Kindervag in 2010, who identified inherent weaknesses

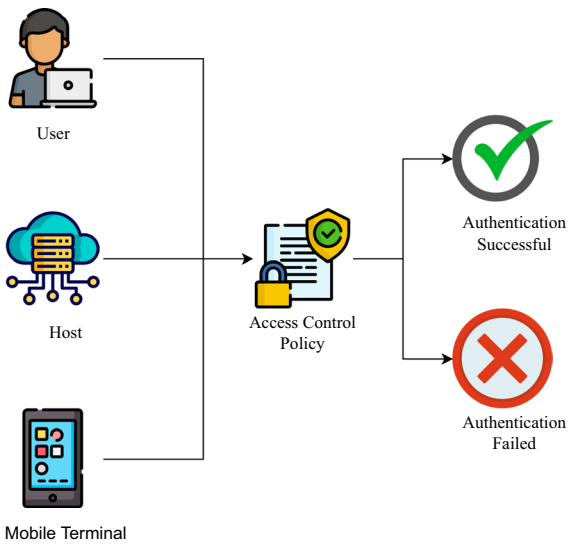


Fig. 1 Traditional access control framework

in traditional perimeter-based security architectures. He pointed out that internal networks, which are typically assumed to be trustworthy, are equally susceptible to security threats, and argued that the notion of implicit trust is fundamentally problematic in modern security systems. Accordingly, he articulated the principle that "trust is the fatal vulnerability of security." Under the Zero Trust paradigm, security is established through strict identity verification, enforcement of the principle of least privilege, and continuous monitoring and validation of access behaviors.

Recent studies have explored Zero Trust security in cloud environments from complementary perspectives. Mani et al. translate Zero Trust segmentation into scalable policy construction and inference for public cloud networks, yet effective generalization and governance in strongly isolated, cross-account, and cross-domain multi-tenant scenarios remain challenging (Mani et al. 2025). Hsieh et al. highlight the limitations of perimeter-based and static Zero Trust approaches in cloud data-plane security and strengthen east–west visibility through robust anomaly detection; however, their focus on detection provides limited support for fine-grained access control enforcement and policy lifecycle management (Hsieh et al. 2024). Grubbs et al. leverage zero-knowledge proofs to enable verifiable policy enforcement over encrypted traffic without content disclosure, aligning with the Zero Trust principle of minimal exposure, but the associated overhead and engineering complexity hinder large-scale cloud deployment (Grubbs et al. 2022). Anjum et al. emphasize that Zero Trust in cloud-centric environments must move beyond simple application migration and gateway authentication to support more expressive security requirements, while leaving end-to-end integration with cloud-native runtimes to future work (Anjum et al. 2022). The comparison between TLBAC and the baseline systems is shown in Table 1.

Table 1 Feature comparison between TLBAC and baseline systems

Feature	TLBAC	Mani	Hsieh	Grubbs
Zero trust	✓	✓	✓	✓
Fine-grained access control	✓	✓	×	×
Identity-based policy	✓	✓	×	×
Lightweight deployment	✓	✓	✓	×
Protocol-layer integration	✓	×	×	×

Design and implementation

This paper proposes TLBAC, a Zero-Trust-based cloud–edge–endpoint access control system using trusted labels. The system embeds trusted labels into network communications to enable fine-grained access control within a cloud–edge–endpoint collaborative architecture. Under the Zero Trust paradigm, every access request undergoes strict identity authentication, is authorized according to the principle of least privilege, and is continuously verified throughout the system lifecycle. In this way, TLBAC provides resilient and adaptive security protection for dynamic and distributed environments. The cloud–edge–endpoint architecture of TLBAC is illustrated in Fig. 2.

In the proposed cloud–edge–endpoint collaborative architecture, each layer is assigned distinct responsibilities and operates in coordination to embody the core principles of Zero Trust. The endpoint layer is represented by the Client Application, which generates trusted labels based on verifiable factors such as user identity and device posture, and embeds them into outgoing TCP traffic to attach verifiable trust information at the source of each access request. The edge layer is embodied by the Edge Decision Gateway, which serves as the policy enforcement point by continuously inspecting incoming traffic, parsing trusted labels, and evaluating each access request against the received policies, thereby ensuring that no request is implicitly trusted. The cloud layer is represented by the Security Cloud Brain, which centrally manages security policies, supports dynamic policy updates, and disseminates these policies to the edge layer for consistent enforcement. Through this coordinated three-layer design, security control is shifted from static network perimeters to identity- and context-driven continuous verification, enabling Zero Trust–compliant access control in dynamic and distributed environments.

The endpoint layer design

The endpoint layer interacts directly with the access subject and is responsible for initiating access requests. It collects user identity information together with supplementary attributes required for identity verification, consolidates them, and generates a trusted labels. This trusted labels is then transmitted to the cloud layer, where the cloud performs authentication and assigns fine-grained access permissions to the access subject with respect to the target resource (access object). The resulting access control policy is subsequently distributed to the edge layer, where enforcement takes place. During subsequent communication, the endpoint layer embeds the trusted labels into outbound TCP packets, enabling the edge layer to make access control decisions in real time based on the embedded labels without requiring additional interaction with the cloud.

The endpoint layer therefore plays a critical role in establishing a secure access context, acting as the origin of trusted identity and behavior information. It primarily consists of two functional components: the trusted labels

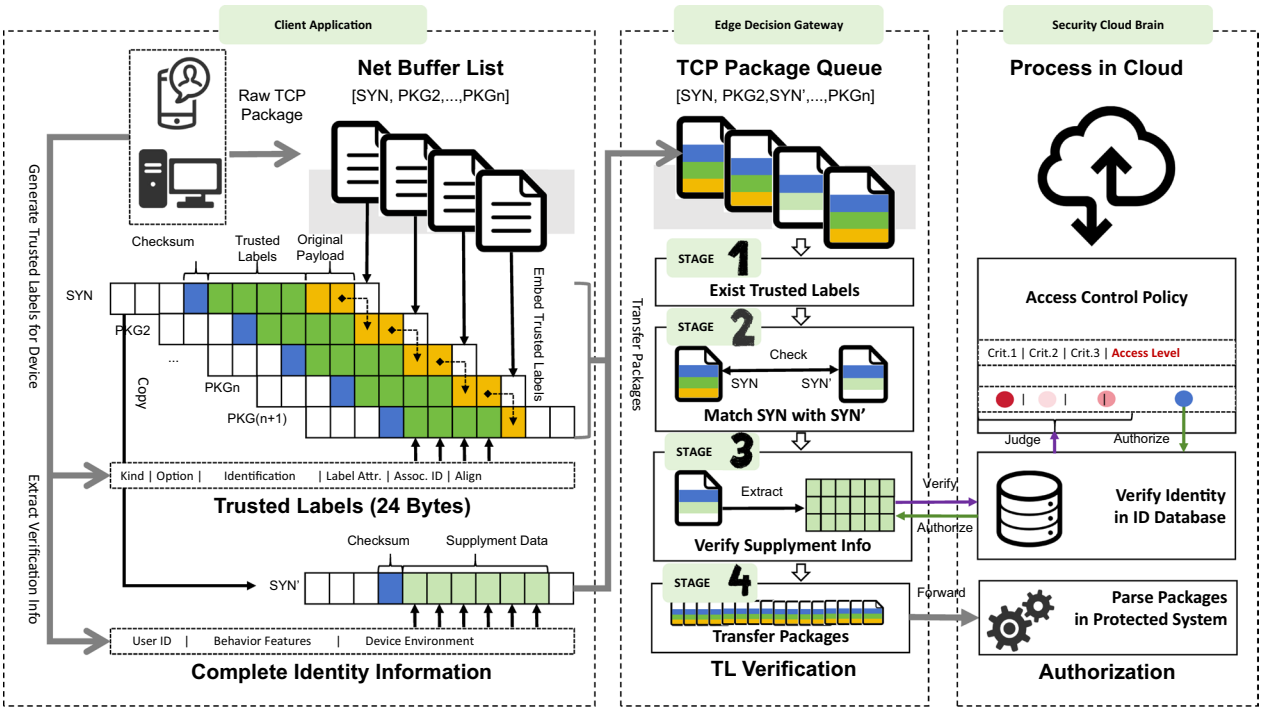


Fig. 2 The cloud-edge-endpoint architecture

generation module and the trusted labels injection module, as illustrated in Fig. 3.

The trusted labels generation module is responsible for constructing the trusted labels, which consists of two components: (i) identity information and (ii) supplementary identity-verification information. The identity information is issued by the cloud, whereas the supplementary verification information is continuously monitored and generated by an application-layer background service running on the endpoint device. The identity information is a 16-byte unique identity credential assigned by the cloud after the client successfully authenticates to the platform. The supplementary identity-verification information consists of behavioral characteristics generated during business access, as well as device environmental attributes.

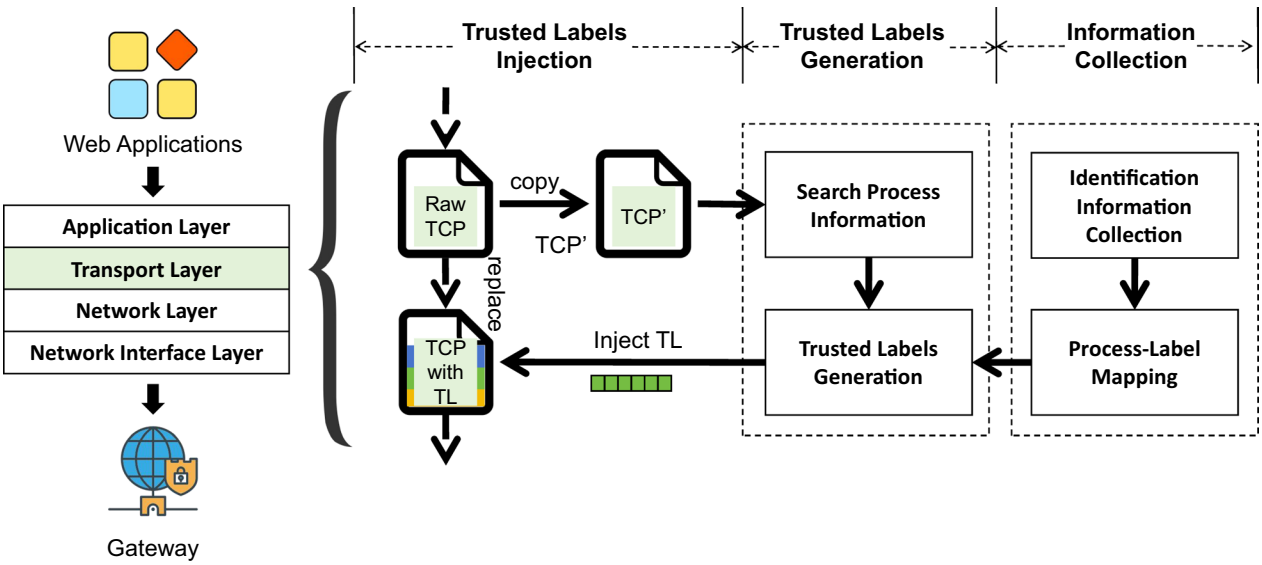


Fig. 3 The flowchart of the traffic control decision algorithm

The IP address serves as the most fundamental element for addressing and routing within the Internet communication architecture, forming the core foundation for end-to-end data transmission. However, the IP address merely denotes the logical location of a network node and lacks intrinsic capabilities for identity authentication and trust verification. In open network environments, adversaries can forge or manipulate IP addresses to conceal their true identities, mislead communication counterparts, or bypass security defenses. The security and verifiability of IP addresses thus constitute a critical prerequisite for constructing a trustworthy network architecture. Given that current Internet communications remain fundamentally based on the IP protocol, the realization of a trustworthy network must be anchored in the IP infrastructure, with trust enhancement mechanisms applied to the existing addressing scheme to ensure authenticity, integrity, and accountability across network entities. This paper implements fine-grained access control through the integration of authentication and authorization trusted labels into IP packets.

(1) *Construction of trusted labels*. Many network operators, by default, discard or sanitize packets containing IP options, primarily because certain historical IP options—such as source routing (LSRR/SSRR)—were once exploited for bypassing, evasion, or amplification attacks. In addition, some security devices, including firewalls and intrusion prevention systems , often treat any packet carrying IP options as anomalous traffic and remove or drop it to mitigate potential risks. In contrast, TCP options are widely supported and relied upon in public networks, with most intermediate devices exhibiting good compatibility with standard TCP options. Therefore, in this study, trusted labels are embedded within the TCP header options to achieve a practical and secure mechanism for trust propagation.

Given that the TCP header can have a maximum length of 60 bytes, and 20 bytes are reserved for the fixed base header, the remaining 40 bytes are available for optional fields. Considering that certain TCP packets (such as SYN packets) occupy part of the option space, to ensure network compatibility and normal operation, empirical testing determined that the size of the trusted labels should not exceed 24 bytes, as shown in Table 2.

Kind field. It represents the type of the TCP option. The value is set to 0x19, indicating that the TCP option corresponds to the Window Scale factor, which is typically used to enlarge the TCP window size in order to support higher bandwidth and larger network latencies. Once this value is specified, the TCP option must also include an Option Length field to define the length of the option. Furthermore, another reason for selecting this value is that some network devices may discard packets containing uncommon Kind values.

Table 2 Trusted labels packet format

1Byte	1Byte	16Byte	2Byte	2Byte	2Byte
Kind Field	Option Length	Identity Identifier	Labels Attribute	Supplementary Information	Supplementary Field

Option length. It specifies the length of the TCP header option.

Identity identifier. It denotes the identity credential provisioned or issued by the cloud, as elaborated in a subsequent section.

Labels attribute. It represents the attribute field within the trusted labels, which can, for example, indicate whether the packet serves as a supplementary information packet for identity verification.

Supplementary information. Owing to the limited space available in the TCP option field, the supplementary information for identity verification cannot be contained within it. Therefore, an additional data packet is employed to transmit the identity verification supplementary information, and this field records the ID attribute of the corresponding packet to be associated.

Supplementary field. It represents the field used for aligning the length of TCP packets.

(2) *Composition of the trusted labels*. The trusted labels comprises identity information and supplementary information for identity verification. *The Identity Identifier* is a 16-byte user identifier, serving as a user credential analogous to a citizen's identity card, and is generated by the cloud. When a user logs in, the cloud issues the corresponding credential; when the user logs out or goes offline, the cloud revokes the credential accordingly. *The Supplementary information* for identity verification includes behavioral characteristics and device context information generated during service access. Specifically, behavioral characteristics refer to process-related information associated with generating network requests, while device context primarily refers to the fingerprint of the current device, as shown in Table 3.

The behavioral characteristics refer to the properties of the process that generates the network access request, including:

- Process identifier ,
- File system path of the process executable,
- Digital signature associated with the executable,
- MD5 hash value of the process executable,
- Execution parameters passed to the process at runtime,

Table 3 Detailed contents of the trusted labels

Name	Details
Identity information	A 16-byte user identifier generated by the cloud
Behavioral feature information	Process identifier
	File system path of the process executable
	Digital signature associated with the executable
	MD5 hash value of the process executable
	Execution parameters passed to the process at runtime
	Parent process that initiates or spawns the current process
	Application window name associated with the process.
	Loaded dynamic-link library modules
Device environment information	Host fingerprint, computed as a SHA-256 hash de- rived from CPU, motherboard, storage, OS, and BIOS identifiers
	Username associated with the operating system user account
	Security Identifier that uniquely identifies user or system entities
	Hostname assigned within the OS or network en- vironment

- Parent process that initiates or spawns the current process,
- Application window name associated with the process,
- Loaded dynamic-link library modules.

The device environmental attributes describe the endpoint fingerprint and include:

- Host fingerprint, computed as a SHA-256 hash derived from CPU, motherboard, storage, OS, and BIOS identifiers,
- Username associated with the operating system user account,
- Security Identifier that uniquely identifies user or system entities,
- Hostname assigned within the OS or network environment.

All of this information is collected at the endpoint layer. An application-layer background service continuously monitors running processes, generating behavioral characteristics—including hash values, digital signatures, parent–child process relationships, absolute executable paths, and window names—as well as device environmental attributes. These data are stored in a `protobuf` structure in the form of key–value mappings, where the key corresponds to the absolute path of the process.

The application layer then transmits this mapping to the network driver layer. When a packet is transmitted, the network driver retrieves the corresponding process and its absolute path based on the source port number. By querying the mapping, the driver obtains behavioral and environmental attributes, which are used as supplementary identity-verification information for constructing the trusted labels.

The trusted labels injection module primarily embeds the trusted labels generated by the trusted labels generation module into TCP packets. Since the maximum available length of the TCP header options field is 40 bytes—and certain TCP packets (such as SYN packets) occupy part of this space—the effective length that can be safely utilized for TCP options, while ensuring normal network functionality, is 24 bytes.

The edge layer design

During the embedding of the trusted labels packet into a TCP packet, it is essential to evaluate whether the increased data volume resulting from the embedding process conforms to the inherent capacity of the TCP packet. This consideration is closely related to two fundamental networking parameters: the Maximum Transmission Unit (MTU) and the Maximum Segment Size (MSS). Therefore, when the TCP packet length exceeds the MTU after embedding the identity information, the packet must be fragmented to ensure proper transmission.

To achieve secure communication, the trusted labels is embedded within a TCP segment. During the data transmission process, two TCP packets are generated through the TCP protocol. Specifically, the trusted labels is encapsulated in Packet A, while Packet B functions as a standard TCP communication packet used for normal data exchange. The two packets are correlated through the checksum and token fields of Packet B. Under ideal conditions, Packet A should arrive at the gateway before Packet B to complete authentication. If Packet B arrives earlier than Packet A, it is temporarily buffered in a queue until Packet A arrives for subsequent verification,

or until the maximum number of retries is reached, in which case the process fails. Once Packet A reaches the gateway, its contents are parsed and validated. Upon successful authentication, Packet B is forwarded to the server; otherwise, Packet B is discarded to prevent unauthorized transmission. The entire transmission and verification process is illustrated in Fig. 4.

At the edge layer, an Edge Decision Gateway is deployed to inspect and identify traffic originating from the endpoint layer. Based on the security policies issued by the cloud layer, the gateway enforces access decisions by either permitting or blocking the traffic. By inspecting trusted labels, the edge layer enables the discovery of network attack threats at the endpoint layer, supports access control decision-making, and performs attack mitigation. The edge layer primarily consists of two components: a traffic label identification module and a traffic control decision module.The traffic label identification module is responsible for extracting trusted labels from access request traffic.

Algorithm 1 Traffic label identification module

Require: Incoming TCP packet stream $P = \{p_1, p_2, \ldots, p_n\}$
Ensure: Trusted labels TL associated with authenticated packets
Initialize waiting queue $Q \leftarrow \emptyset$ {temporarily stores unmatched B packets}
Initialize label store $S \leftarrow \emptyset$ {stores trusted labels from A packets}
for each incoming packet p in P **do**
  if $p.type == A$ **then** {A packet carries trusted labels}
    $TL \leftarrow extract_label(p)$
    store $(Checksum(p), TL)$ in S
    if $\exists\ b \in Q$ such that $Checksum(b) = Checksum(p)$ **then**
      authenticate(b, TL)
      remove b from Q
    end if
  else if $p.type == B$ **then** {B packet carries normal authentication data}
    if $\exists\ TL \in S$ such that $Checksum(TL) = Checksum(p)$ **then**
      authenticate(p, TL)
    else
      enqueue(Q, p) {possible packet reordering}
    end if
  end if
  for each buffered packet $b \in Q$ **do**
    if b has waited for more than N subsequent arrivals **then**
      discard b from Q
    end if
  end for
end for

The traffic label identification module are then stored and subsequently used by the traffic control decision module to make access decisions, as shown in Algorithm 1.The primary objective of the Traffic Label Identification Module is to associate and authenticate type-A packets carrying trusted security labels with type-B packets that contain authentication requests, thereby ensuring the reliability of access-control

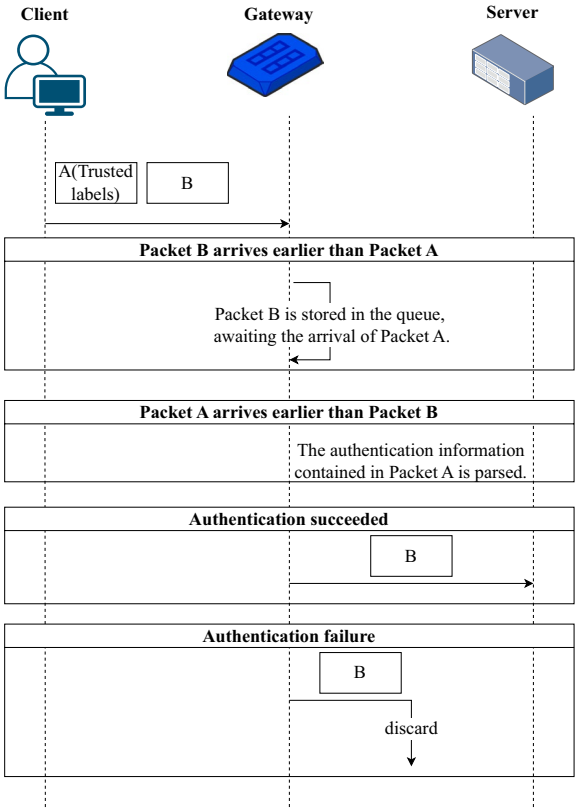


Fig. 4 Transmission of the trusted labels

decisions in network transmission environments. Upon receiving a continuous TCP packet stream, the module applies different processing logic depending on the detected packet type. When a type-A packet is identified, the system immediately extracts the trusted labels, which includes security metadata such as user identity, device attributes, and behavior information. The system then uses a checksum as a correlation key to determine whether a pending type-B packet with the same identifier exists in the queue. If a match is found, the trusted labels is bound to the type-B packet and authentication is completed; otherwise, the trusted labels is stored for subsequent matching. When a type-B packet arrives, the system also checks whether a previously stored trusted labels with a matching checksum exists; if so, authentication is performed immediately. If no trusted labels is available, the type-B packet is temporarily stored in the queue and waits for an associated type-A packet to arrive. To prevent resource exhaustion caused by abnormal traffic or malicious attacks, the system enforces a maximum waiting window N for unmatched type-B packets, after which they are discarded. This mechanism supports robust handling of out-of-order packet arrival, ensuring that authentication decisions

are not affected by network jitter or multipath transmission. Moreover, by leveraging checksum-based correlation, trusted labels can only be associated with authentication data from the same communication context, reducing mismatching risks and improving authentication accuracy. Through these mechanisms, the module securely binds trusted labels to authentication requests and enables reliable authentication, thereby providing a trustworthy foundation for fine-grained access control.The flowchart of the Traffic Label Identification algorithm is shown in Fig. 5.

The traffic control decision module determines whether the requesting entity is permitted to access the target resource based on the trusted labels extracted and recorded by the traffic label identification module, in conjunction with the access control policies issued by the cloud layer. If the request conforms to the policy—i.e., the trusted labels is deemed valid—the traffic is allowed. Conversely, if the request violates the policy or the trusted labels is considered untrustworthy, the traffic is blocked, as shown in Algorithm 2.

Algorithm 2 Traffic Control Decision Module

Require: Incoming SYN request packet p
Ensure: Permit or block the connection
Extract packet identifier $id \leftarrow p.id$
Retrieve trusted labels TL corresponding to id from label storage
Retrieve current policy set $R \leftarrow AccessControlPolicy$
{Case 1: trusted labels missing}
if TL is NULL **then**
  send_reset_packet(p)
  log_event("missing trusted labels", id)
  return
end if
{Case 2: label expired, malformed, or incomplete}
if TL is expired **OR** malformed **OR** incomplete **then**
  send_reset_packet(p)
  log_event("invalid trusted labels", id)
  return
end if
{Case 3: policy set unavailable or outdated}
if R is NULL **OR** stale **then**
  send_reset_packet(p)
  log_event("policy unavailable or stale", id)
  return
end if
{Case 4: evaluate policy rules}
for each policy rule r in R **do**
  if $TL.identity$ matches $r.identity$ **and** $TL.behavior_info$ matches $r.behavior_constraints$ **and** $TL.device_info$ matches $r.device_constraints$ **then**
    permit_traffic(p)
    log_event("connection permitted", id)
    return
  end if
end for
{Case 5: no matching policy rule}
send_reset_packet(p)
log_event("no matching policy", id)

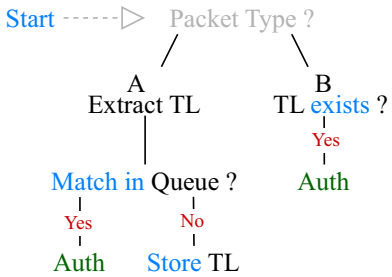


Fig. 5 The flowchart of the traffic label identification algorithm

The traffic control decision module is responsible for determining whether a network connection request is allowed based on the trusted labels previously obtained by the Traffic Label Identification Module. When an incoming TCP `SYN` request packet arrives at the edge gateway, the module first extracts the packet identifier (e.g., flow ID or five-tuple) and attempts to retrieve the corresponding trusted labels *TL* from the label storage. If no trusted labels exists, the packet is considered unverified and the module immediately terminates the connection by sending a TCP `RST` packet, thereby enforcing a "default deny" principle consistent with Zero Trust security requirements.

If a trusted labels is found, the module performs policy-based access evaluation. The edge gateway maintains an access control policy set, dynamically synchronized from the cloud layer. Each policy rule consists of three constraints: (*i*) identity constraint, (*ii*) behavior constraint, and (*iii*) device constraint. For each policy rule, the module compares the trusted labels fields against the rule specification. A connection is permitted only when all constraints are simultaneously satisfied. Otherwise, the module rejects the access request by sending a TCP `RST` packet. This mechanism implements a multi-dimensional authorization model, ensuring that only users with trusted identity, legitimate behavior intent, and compliant device posture are granted access. As a result, the proposed module achieves fine-grained and trustworthy access control while minimizing the latency of decision execution through edge-side evaluation.The flowchart of the Traffic Control Decision algorithm is shown in Fig. 6.

The cloud layer design

In the cloud layer, a Security Cloud Brain is deployed, which may consist of a single computer, a server, or a server cluster. It is responsible for performing identity verification and intelligent analysis on access requests initiated from the endpoint layer, generating corresponding security policies that are distributed to the

edge layer. Additionally, it conducts auditing and traceability for anomalous endpoints. Automatic access control policies are generated based on the trusted labels and behavioral classifications of the access subjects. The degree of leniency in access authorization decreases as the behavioral level increases, and the generated access control policies are subsequently distributed to the Edge Decision Gateway. This paper classifies behaviors according to four evaluation dimensions: resource importance, resource sensitivity, behavior sensitivity, and environmental risk. According to the relevant provisions of the national standard GB/T 22239-2019, Information Security Technology — Baseline Requirements for Cybersecurity Classified Protection (Gb 2019), critical business resources are required to be protected at a higher security level. In this paper, resource importance is divided into two levels: A-1 denotes critical resources, while A-2 represents general resources. According to the relevant provisions of the national standard GB/T 35273-2020, Information Security Technology — Personal Information Security Specification (Gb 2020), the processing of sensitive data requires a higher level of protection. In this paper, resource sensitivity is classified into two levels: B-1 represents internal data, while B-2 denotes public data. According to the relevant provisions of the national standard GB/T 20986-2023, Guidelines for Classification and Grading of Cybersecurity Incidents (Gb 2023), sensitive operations are classified as higher-level incidents and should therefore be assigned an elevated risk level. In this paper, behavior sensitivity is divided into two levels: C-1 represents sensitive behaviors, while C-2 represents normal behaviors. According to the relevant provisions of the national standard GB/T 43696-2024, Zero Trust — Reference Architecture (Gb 2024), untrusted environments should enforce stricter access authorization and adopt a least-privilege strategy. In this paper, environmental risk is classified into two levels: D-1 represents public network environments, while D-2 represents internal network environments.The behavioral classification table is presented in Table 4.

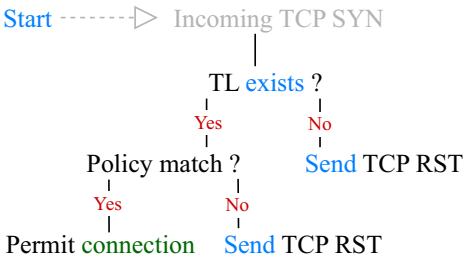


Fig. 6 The flowchart of the traffic control decision algorithm

According to the requirements specified in the corresponding national standards, this paper assigns weights to the evaluation dimensions as follows: resource importance ($w_A = 0.30$), resource sensitivity ($w_B = 0.20$), behavior sensitivity ($w_C = 0.35$), and environmental risk ($w_D = 0.15$).

The final risk score is calculated according to the formula:

$$R = w_A r_A + w_B r_B + w_C r_C + w_D r_D \tag{1}$$

where w_A, w_B, w_C, and w_D represent the weights of each evaluation dimension, and r_A, r_B, r_C, and r_D denote their corresponding assigned values.

According to the computed score R, the risk level is categorized into four grades:

$$\text{Level} = \begin{cases} \text{I}, & R < 0.30, \\ \text{II}, & 0.30 \le R < 0.55, \\ \text{III}, & 0.55 \le R < 0.75, \\ \text{IV}, & R \ge 0.75. \end{cases} \tag{2}$$

The corresponding score is computed using Equation (1), and the final classification results are presented in Table 5.

Table 4 Behavioral classification table

Dimension	Meaning	Risk score	Rationale
A: Resource importance	A-1: Critical resources	1.0	GB/T 22239-2019 *Information Security Technology—Baseline Requirements for Cybersecurity Classified Protection*
	A-2: General resources	0.4	
B: Resource sensitivity	B-1: Internal data	1.0	GB/T 35273-2020 *Information Security Technology—Personal Information Security Specification*
	B-2: Public data	0.3	
C: Behavior sensitivity	C-1: Sensitive behaviors	1.0	GB/T 20986-2023 *Guidelines for Classification and Grading of Cybersecurity Incidents*
	C-2: Normal behaviors	0.2	
D: Environmental risk	D-1: Public network environments	0.9	GB/T 43696-2024 *Zero Trust – Reference Architecture*
	D-2: Internal network environments	0.3	

The access control policy of the Security Cloud Brain is applied after the Edge Decision Gateway identifies trusted labels. Trusted users are then categorized by roles and assigned hierarchical access privileges. Based on these privilege levels, trusted users are permitted to access different types of resources, thereby enabling fine-grained access control.The detailed implementation process of the access control policy is illustrated in Fig. 7.

Access control policies are generated according to the identities and behavioral classifications of the access subjects and are then distributed to the Edge Decision Gateway. The policies operate in two modes: Detection Mode and Protection Mode. In *Detection Mode*, the system performs traffic inspection and threat alerting without intercepting the traffic. In *Protection Mode*, the system both intercepts malicious traffic and issues alerts to ensure secure network operations.

The traffic control decision module determines whether the requesting entity is permitted to access the target resource based on the trusted labels extracted and recorded by the traffic label identification module, in conjunction with the access control policies issued by the cloud layer. If the request conforms to the policy—i.e., the trusted labels is deemed valid—the traffic is allowed. Conversely, if the request violates the policy or the trusted labels is considered untrustworthy, the traffic is blocked, as shown in Algorithm 2.

Evaluation

In this section, experiments are conducted to validate the advantages of *TLBAC* and to address the following key research questions:

- *RQ*1: Does embedding trusted labels into TCP packets introduce any impact during data transmission?
- *RQ*2: What is the system overhead incurred by using TLBAC?
- *RQ*3: For a protected system known to contain numerous threats, can TLBAC effectively defend against hacker attacks?
- *RQ*4: In the context of multi-tenant deployments, how do the performance scalability and policy isolation properties of TLBAC hold up under increasing tenant load?

Experimental setups

TLBAC is implemented in the Go programming language. For the cloud layer and edge layer deployments, we use virtual machines created with Proxmox VE as the execution environment. Each VM is configured with an AMD R9-6900HX 3.3 GHz CPU (allocated 4 cores), 4 GB of memory, and runs Ubuntu 24.04 Server as the operating system.

For the endpoint layer, the execution environment is a physical smartphone device—an Honor series phone (model ELP-AN00), equipped with a Qualcomm Snapdragon 8s Gen 3 processor, 16 GB of RAM, 512 GB of storage, and running Android 16 with MagicOS 10.0.

Feasibility analysis of trusted labels(RQ1)

Inserting trusted labels into the transport-layer protocol requires extending and modifying the original TCP packet format. To assess whether such structural modifications affect transmission reliability, and to determine

Table 5 Classification table of levels

No.	A	B	C	D	Score	Levels
1	A-1	B-1	C-1	D-1	0.985	IV
2	A-1	B-1	C-1	D-2	0.895	IV
3	A-1	B-1	C-2	D-1	0.705	III
4	A-1	B-1	C-2	D-2	0.615	III
5	A-1	B-2	C-1	D-1	0.845	IV
6	A-1	B-2	C-1	D-2	0.755	IV
7	A-1	B-2	C-2	D-1	0.565	III
8	A-1	B-2	C-2	D-2	0.475	II
9	A-2	B-1	C-1	D-1	0.805	IV
10	A-2	B-1	C-1	D-2	0.715	IV
11	A-2	B-1	C-2	D-1	0.525	II
12	A-2	B-1	C-2	D-2	0.435	II
13	A-2	B-2	C-1	D-1	0.665	III
14	A-2	B-2	C-1	D-2	0.575	III
15	A-2	B-2	C-2	D-1	0.385	II
16	A-2	B-2	C-2	D-2	0.295	I

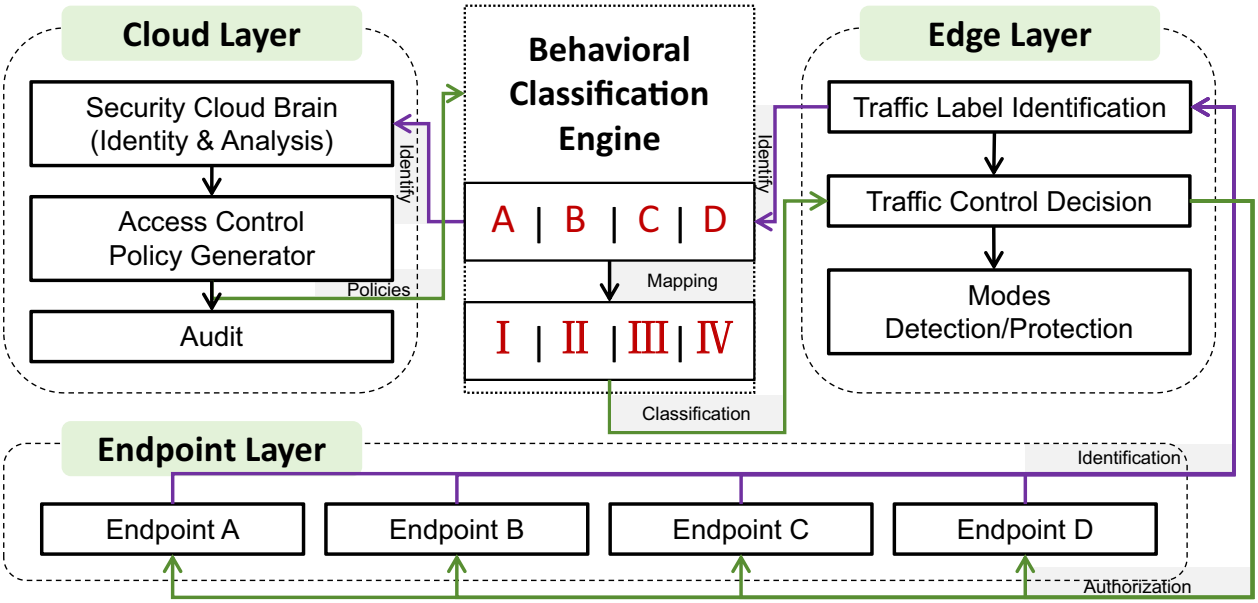


Fig. 7 The access control policy flowchart

whether the modified TCP packets can still stably carry trusted labels and complete communication procedures under complex network conditions, this study designs and conducts a targeted feasibility verification experiment.

For the experimental environment, an Android device is used as the endpoint terminal (client). A representative LTE mobile network environment is emulated, incorporating an average round-trip latency of 100 ms, network jitter of ±50 ms, and a random packet loss rate of 10%, thereby approximating the fluctuation and uncertainty typical of real cellular networks. Under this network configuration, the system constructs HTTP data packets carrying trusted labels, with each packet containing a payload of 100 KB.

To comprehensively analyze the impact of trusted labels insertion on TCP reliability mechanisms, the experiment transmits packet sequences of varying sizes, including a single group (1 group), medium-scale sequences (10 and 100 groups), and a large continuous sequence (200 groups). By observing and recording TCP retransmission behavior under these progressively increasing transmission loads, the experiment evaluates whether the insertion of trusted labels introduces additional transmission errors, handshake irregularities, or retransmission overhead, thereby determining whether the modified TCP packets maintain stable and reliable communication performance even under adverse network conditions.

As shown in Table 6, four groups of experiments were conducted, in which the client continuously transmitted 1, 10, 100, and 200 groups of data packets, respectively. In Experiment 3, although 100 groups of data packets were sent, the total number of TCP segments transmitted reached 204, indicating the presence of TCP segmentation and retransmission. The number of TCP segments received at the edge layer matches exactly with the number transmitted from the endpoint, demonstrating that no data loss occurred during transmission. As the number of data packet groups increases, the total transmitted bytes grow linearly, reflecting that the insertion of trusted labels does not alter the linear scaling behavior of data size. Furthermore, the total number of bytes received at the edge layer is identical to that sent from the endpoint, confirming the integrity and consistency of the transmitted data.

As illustrated in Table 7, when the data volume is small (Experiments 1 and 2), TCP retransmissions are minimal (1 and 10 occurrences, respectively). In contrast, larger data volumes (Experiments 3 and 4) result in higher retransmission counts (141 and 294 occurrences). These retransmissions, however, are not caused by the insertion of trusted labels; rather, they are induced by the LTE network emulation settings, which include a 10% packet loss rate, network latency, and jitter.

The experimental results demonstrate that even under adverse LTE network conditions—characterized by high latency, noticeable jitter, and substantial packet loss—the TCP packets augmented with trusted labels are still able to maintain stable and reliable data transmission. The number of TCP segments and total bytes observed at the endpoint and edge remain perfectly consistent,

Table 6 TCP transmission statistics for trusted labels feasibility evaluation

No.	Data packet	Endpoint TCP segments sent	Edge TCP segments received	Bytes sent at endpoint	Bytes received at edge
1	1	3	3	308,096	308,096
2	10	21	21	3,080,956	3,080,956
3	100	204	204	30,536,859	30,536,859
4	200	407	407	61,061,024	61,061,024

Table 7 TCP retransmission behavior across different data volumes

No.	Data packet	Retransmitted packet (trusted labels)
1	1	1
2	10	10
3	100	141
4	200	294

indicating that trusted labels insertion does not introduce additional segmentation errors or payload corruption. Moreover, the number of retransmissions increases linearly with the data volume, aligning with the expected behavior of the network model and showing no signs of abnormal retransmission patterns.

Therefore, it can be concluded that TCP, even after being extended to carry trusted labels, retains its inherent reliability mechanisms and can stably transport trusted-labels–enhanced packets under realistic and challenging network conditions.

Performance overhead assessment for TLBAC (RQ2)

To evaluate the performance overhead introduced by TLBAC during practical operation, this study defines and examines three representative system modes. The first is the Direct Mode, in which TLBAC remains completely passive during client communication and file access, performing no traffic analysis or feedback. The second is the Detection Mode, where TLBAC actively inspects network traffic generated during client access, conducting trusted labels parsing and security analysis without enforcing any blocking actions. The third is the Protection Mode, in which TLBAC not only performs traffic detection but also applies security policies to determine whether suspicious or malicious traffic should be blocked, thereby simulating the system's behavior under full protection. Within the Linux operating system, iptables functions as a classic firewall system for configuring packet-filtering rules in the kernel-level Netfilter framework. It provides support for filtering, forwarding, modifying, and logging inbound and outbound network packets, thereby making it one of the core components of Linux network security. Accordingly, this chapter adopts iptables as the baseline system for evaluating the performance of TLBAC.

To systematically evaluate the system performance under different data-volume conditions, a protected test environment was established, in which a test set consisting of ten files of different sizes was constructed, namely 10 MB, 50 MB, 100 MB, 300 MB, 500 MB, 800 MB, 1 GB, 1.2 GB, 1.5 GB, and 2 GB. The experiments examined not only the three operating modes of TLBAC, but also introduced the classic Linux firewall system iptables as a baseline for comparative evaluation. During the tests, the client sequentially accessed the above files under each experimental mode, while the system simultaneously recorded real-time CPU utilization and memory usage. By comparing the differences in resource consumption between the three TLBAC modes and the iptables mode across different file sizes, the computational and memory overhead introduced by TLBAC's detection and interception mechanisms can be comprehensively evaluated, thereby further analyzing its overall performance impact in practical deployment scenarios. To improve the reliability of the experimental results, each group of tests was repeated three times, and the average value was taken as the final result. Subsequently, the raw monitoring data were cleaned and statistically analyzed using the pandas library, and the average value, maximum value, and fluctuation range of each performance indicator were calculated. Table 8 presents the comparison results of the key performance indicators between the three operating modes of TLBAC and the iptables baseline system.

As shown in the Table 8, the four operating modes exhibit noticeable differences in CPU overhead, whereas the differences in memory usage remain relatively small. Specifically, the average CPU utilization of Direct Mode and Iptables Mode is 0.20% and 0.20%, respectively, both of which are at a very low level, indicating negligible computational overhead. By contrast, the average CPU utilization increases to 0.39% in Detection Mode and 0.44% in Protected Mode, demonstrating that the detection and protection mechanisms introduced by TLBAC incur additional computational cost. Among all modes,

Protected Mode exhibits the highest average and peak CPU utilization, reaching 0.44% and 1.85%, respectively, which indicates that this mode involves more processing operations during execution and therefore results in higher instantaneous and overall computational overhead.

In terms of stability, Iptables Mode achieves the smallest CPU standard deviation, at only 0.03, and also presents the narrowest 95% confidence interval, indicating the most stable execution behavior among all modes. In comparison, the CPU standard deviations of Detection Mode and Protected Mode increase to 0.21 and 0.26, respectively, suggesting that the introduction of detection and interception mechanisms leads to greater fluctuations in CPU load. As illustrated in Fig. 8, when Iptables Mode is taken as the baseline, the absolute increases in average CPU utilization are only 0.19 and 0.24 percentage points, respectively. Although the relative increase appears noticeable, the average CPU utilization in both modes remains below 0.5% in absolute terms, indicating that the additional computational overhead remains well within a controllable range.

A further examination of the fluctuation characteristics reveals that the curves corresponding to Detection Mode and Protected Mode exhibit more frequent stage-wise variations during the test, with Protected Mode showing the most pronounced fluctuation. This suggests that TLBAC introduces additional instantaneous computational load when performing access detection, policy matching, and security interception. Meanwhile, the 95% confidence intervals of the two modes are [0.370, 0.418] and [0.407, 0.469], respectively, both of which remain relatively concentrated. This observation indicates that, although the overhead of TLBAC is higher than that of the iptables baseline, it does not lead to sustained high CPU load. Therefore, the performance cost introduced by TLBAC can be regarded as limited and acceptable in practical deployment scenarios.

Figure 9, together with Table 8, indicates that the system maintains a relatively stable level of memory utilization across different operating modes, with only

Table 8 Performance indicator comparison under four operating modes

Operating mode	Avg CPU(%)	CPU std	CPU 95% CI	max CPU(%)	Avg mem(%)	Mem std	Mem 95% CI	Time(s)
Direct	0.20	±0.13	[0.185, 0.219]	1.12	19.48	±0.04	[19.47, 19.48]	228
Detection	0.39	±0.21	[0.370, 0.418]	1.62	19.51	±0.13	[19.50, 19.53]	300
Protected	0.44	±0.26	[0.407, 0.469]	1.85	19.49	±0.10	[19.47, 19.50]	266
Iptables	0.20	±0.03	[0.198, 0.206]	0.25	19.44	±0.13	[19.42, 19.45]	196

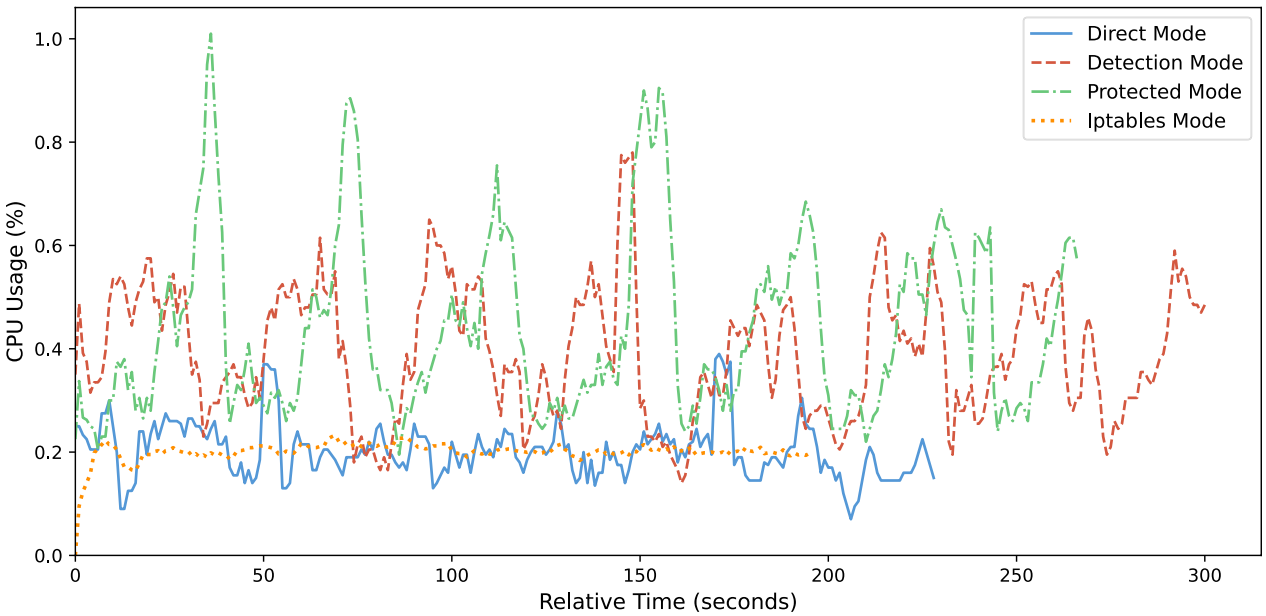


Fig. 8 CPU usage comparison across four modes

minor differences among them. Specifically, the average memory utilization in Direct Mode, Detection Mode, Protected Mode, and iptables Mode is 19.48%, 19.51%, 19.49%, and 19.44%, respectively. Among these, Detection Mode exhibits the highest average value, but it is only 0.03 percentage points higher than that of Direct Mode. Similarly, Protected Mode exceeds Direct Mode by only 0.01 percentage points. Even when compared with the baseline system, iptables, the average memory utilization of Detection Mode and Protected Mode is higher by only 0.07 and 0.05 percentage points, respectively. These results indicate that enabling the detection and protection functionalities of TLBAC does not introduce significant additional memory overhead.

From the perspective of fluctuation, the standard deviations of memory utilization in the four modes are 0.04, 0.13, 0.10, and 0.13, respectively, all of which remain at relatively low levels. This suggests that no evident memory jitter or sustained growth occurred throughout the experiments. Furthermore, the 95% confidence intervals for the four modes are [19.47, 19.48], [19.50, 19.53], [19.47, 19.50], and [19.42, 19.45], respectively. The narrow ranges of these intervals further confirm the stability of memory utilization. Although the maximum memory utilization in Protected Mode reaches 19.70%, which is slightly higher than that in the other modes, neither its average value nor its overall fluctuation range increases significantly, indicating that this peak is more likely a transient local variation rather than a persistent resource burden. Overall, whether the system performs basic data forwarding or more complex operations such as trusted-label parsing, traffic inspection, and security policy enforcement, TLBAC is able to maintain stable and predictable memory usage, demonstrating its feasibility for deployment in resource-constrained environments.

As shown in Fig. 10, the comparative performance results reveal noticeable differences among the four operating modes in terms of CPU overhead, memory usage, and execution time. In terms of average CPU utilization, Direct Mode and iptables Mode both remain at 0.20%, indicating that the system incurs very low computational overhead in the absence of advanced security processing. By contrast, the average CPU utilization rises to 0.39% in Detection Mode and 0.44% in Protected Mode, suggesting that TLBAC introduces additional computational cost when performing label parsing, traffic inspection, and policy enforcement. Similarly, the maximum CPU utilization reaches 1.12%, 1.62%, 1.85%, and 0.25% in Direct Mode, Detection Mode, Protected Mode, and iptables Mode, respectively. Although Protected Mode exhibits the highest instantaneous CPU peak, all peak values remain below 2%, indicating that the overall processor overhead is still limited.

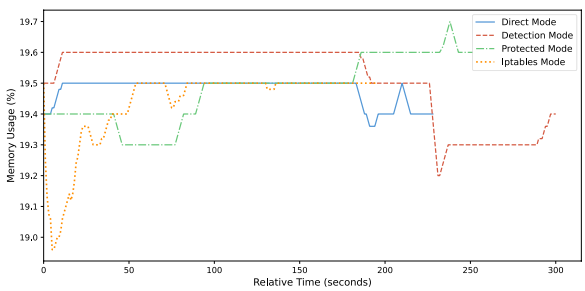


Fig. 9 Memory usage comparison across four modes

In terms of memory consumption, the average memory utilization of the four modes is 19.48%, 19.51%, 19.49%, and 19.44%, respectively, with a maximum difference of only 0.07 percentage points. This result indicates that the detection and protection mechanisms of TLBAC have a negligible impact on memory usage and that the system maintains stable memory behavior across all modes. Regarding total execution time, iptables Mode, Direct Mode, Protected Mode, and Detection Mode require 196 s, 228 s, 266 s, and 300 s, respectively. Detection Mode incurs the longest execution time, implying that continuous inspection imposes the greatest impact on overall efficiency. Although Protected Mode shows the highest average CPU utilization, its execution time is shorter than that of Detection Mode, suggesting that its computational overhead is more concentrated in specific security-processing stages rather than being uniformly sustained throughout execution. Overall, while TLBAC introduces additional CPU and time overhead when its detection and protection mechanisms are enabled, its memory consumption remains comparable to that of the iptables baseline, and its overall resource usage stays at a low level, demonstrating good practical deployability.

Figure 11 shows the variation in average latency across different operating modes under varying concurrency levels. Overall, as the concurrency level increases from 10 to 100, the average latency in all four modes exhibits an approximately linear growth trend, indicating that system latency increases steadily with workload and remains highly predictable. In Direct Mode, Detection Mode, and iptables Mode, the latency curves are almost identical. Specifically, at concurrency levels of 10, 30, 50, 80, and 100, the average latency increases from approximately 2.6 ms to about 34.0 ms. This suggests that the introduction of the detection mechanism does not significantly increase system access latency, and that its performance remains comparable to that of Direct Mode and the iptables baseline.

By contrast, Protection Mode exhibits slightly higher average latency than the other three modes, with the

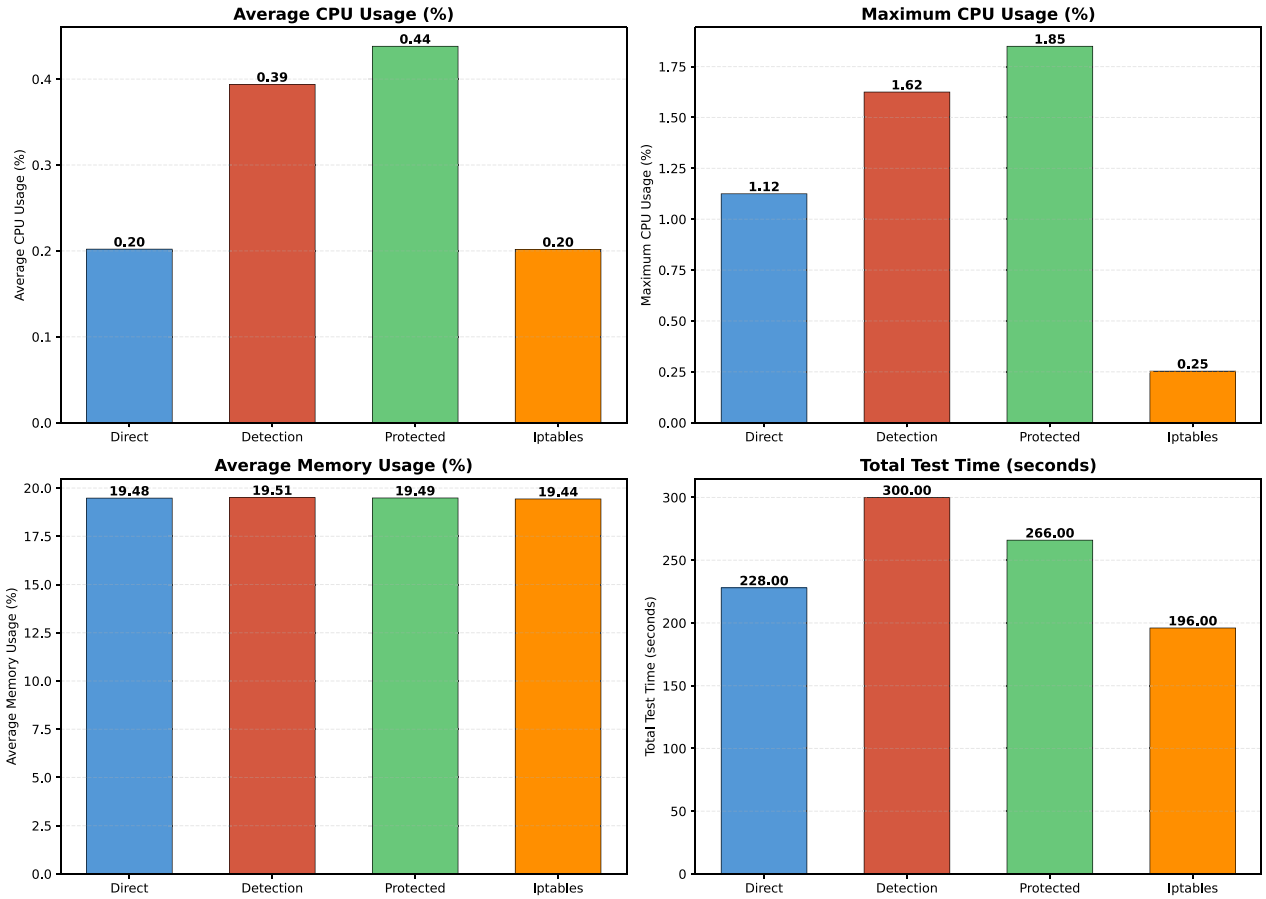


Fig. 10 Performance metrics comparison

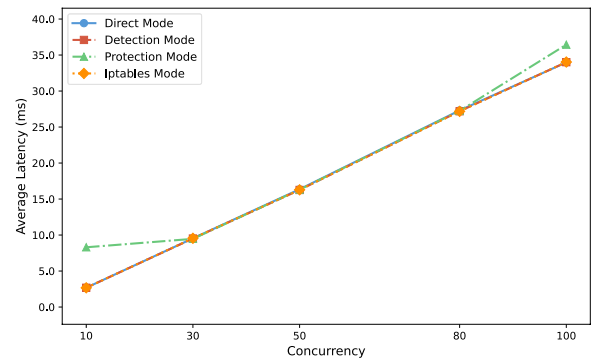


Fig. 11 Comparison of average latency between TLBAC and iptables under different concurrency levels

difference being more noticeable under low and high concurrency. In particular, at a concurrency level of 10, the average latency in Protection Mode is approximately 8.3 ms, significantly higher than the 2.6 ms observed in the other modes. At concurrency levels of 30, 50, and 80, its latency is approximately 9.6 ms, 16.4 ms, and 27.3 ms, respectively, showing only marginal differences from the other modes. When the concurrency level reaches 100, the average latency increases to about 36.4 ms, which is approximately 2.4 ms higher than that of the other modes. These results indicate that the protection mechanism introduces a certain latency overhead when performing more comprehensive security decision-making, policy matching, and interception operations. However, this overhead remains limited and does not lead to abnormal latency growth. Overall, the detection function of TLBAC has a negligible impact on latency, while the protection function introduces only modest additional overhead, demonstrating that the proposed system can maintain favorable latency performance and practical deployability while providing enhanced security protection.

Figure 12 shows the throughput performance of different operating modes under varying concurrency levels. Overall, the system maintains a high throughput across all four modes, ranging from 2840 to 2962 requests/s, indicating strong request-processing capability regardless of whether the detection or protection

mechanisms are enabled. The throughput curves of Direct Mode, Detection Mode, and iptables Mode are closely aligned, remaining within 2920–2962 requests/s across all tested concurrency levels. This indicates that the detection mechanism of TLBAC introduces little to no observable degradation in throughput and performs comparably to both Direct Mode and the iptables baseline.

In contrast, Protection Mode exhibits slightly larger throughput variation. Its throughput is approximately 2840 requests/s at a concurrency level of 10, increases to about 2945 requests/s at 30, and then gradually declines to 2917 requests/s at 100. This suggests that the protection mechanism introduces a certain processing overhead when executing more comprehensive security decision-making, policy matching, and interception operations. However, the throughput remains above 2900 requests/s under most concurrency levels, and the overall deviation from the other modes is limited. Therefore, while the protection function incurs a moderate throughput penalty, the proposed system still maintains favorable throughput performance and practical deployability while providing enhanced security protection.

Effectiveness of TLBAC in mitigating attacks on threat-prone systems (RQ3)

To evaluate the security effectiveness of TLBAC in high-threat environments, this study constructs a protected system that incorporates a diverse set of known vulnerabilities, thereby simulating realistic hacker attack scenarios. The system is subjected to attacks under two conditions—with TLBAC enabled and disabled—to comparatively assess the defensive capabilities of TLBAC. This design enables a systematic analysis of TLBAC's performance in mitigating cyberattacks and validates its effectiveness in defending threat-prone systems.

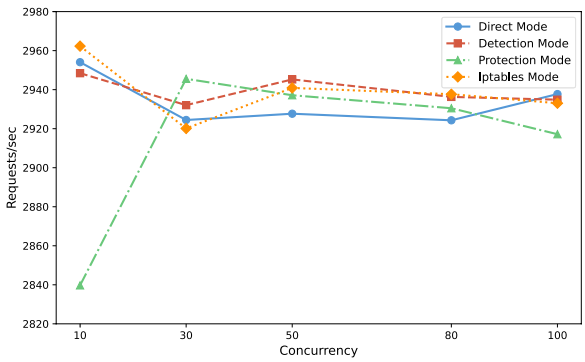


Fig. 12 Throughput comparison across different operating modes under varying concurrency levels

Through this experimental setup, the study not only reproduces realistic high-risk attack conditions but also enables a comprehensive evaluation of TLBAC's defensive capability across diverse and multifaceted threat scenarios. Table 9 summarizes the 30 selected vulnerabilities used in the attack simulation experiment. For each vulnerability, the table reports its CVE identifier, associated CWE weakness category, affected software component and version, attack method, and whether the attack was detected and successfully intercepted by TLBAC.

Based on the vulnerability set summarized in Table 9, this study constructs an attack simulation environment that encompasses a wide range of attack vectors, software components, and version variations in order to systematically evaluate the detection and defense capabilities of TLBAC under realistic threat conditions. The selected vulnerabilities span critical infrastructure and mainstream application frameworks, including DNS, SNMP, BIND, Memcached, Log4j 2, Spring, ActiveMQ, Jenkins, Redis, GitLab, MinIO, Symfony, Django, Fortinet, and ThinkPHP. The corresponding CWE categories involve diverse security weaknesses such as buffer overflow (CWE-122), null pointer dereference (CWE-476), weak cryptography (CWE-328), path traversal (CWE-22), insecure deserialization (CWE-502), access control failures (CWE-284), authentication bypass (CWE-287), and cross-site scripting (CWE-79). These vulnerabilities are commonly found in real-world systems and possess high exploitation value, making them representative of the complex threat landscape faced by modern computing environments.

Experimental results demonstrate that TLBAC successfully detected and blocked all attacks across more than thirty vulnerability cases involving different components, versions, and exploitation techniques, without a single instance of missed detection or erroneous allowance. Each attack was accurately classified as Detected and subsequently Blocked, exhibiting strong consistency and stability. This outcome illustrates that TLBAC's trusted-labels mechanism and cross-layer inspection logic are capable of capturing subtle behavioral signals embedded in malicious traffic, thereby enabling unified detection across heterogeneous protocols, traffic patterns, and exploitation paths. Moreover, by embedding trusted labels into the transport layer, TLBAC can identify and intercept potential threats before vulnerability-triggering logic is executed at the application layer, effectively achieving earlier-stage disruption of attack chains.

To further validate the security capability of TLBAC, ten common cybersecurity attack methods were selected for experimental evaluation, including Distributed Denial of Service (DDoS) attacks, phishing, Man-in-the-Middle

Table 9 Analysis of exploited vulnerabilities and resulting defensive performance in the attack simulation

Index	CVE ID	CWE type	Component name	Affected versions	Before protection	After protection
1	CVE-2020-25681	CWE-122	Dnsmasq	< 2.83	Detected	Blocked
2	CVE-2022-44793	CWE-476	Net-SNMP	5.4.3 - 5.9.3	Detected	Blocked
3	CVE-2024-3596	CWE-328	FreeRADIUS	Multiple Versions	Detected	Blocked
4	CVE-2025-40778	CWE-349	BIND 9	9.11.x - 9.21.x	Detected	Blocked
5	CVE-2022-48571	CWE-406	Memcached	1.6.7	Detected	Blocked
6	CVE-2021-44228	CWE-502	Log4j 2	2.0 - 2.14.1	Detected	Blocked
7	CVE-2022-22965	CWE-94	Spring	5.3.0 - 5.3.17	Detected	Blocked
8	CVE-2023-46604	CWE-502	ActiveMQ	< 5.18.3	Detected	Blocked
9	CVE-2024-23897	CWE-22	Jenkins	< 2.442	Detected	Blocked
10	CVE-2022-0543	CWE-269	Redis	Debian/Ubuntu	Detected	Blocked
11	CVE-2021-22205	CWE-78	GitLab	11.9 - 13.10	Detected	Blocked
12	CVE-2023-28432	CWE-200	MinIO	RELEASE < 2023	Detected	Blocked
13	CVE-2025-66022	CWE-78	FACTION Framework	< 1.7.1	Detected	Blocked
14	CVE-2025-65881	CWE-79	Zoo Management System	v1.0	Detected	Blocked
15	CVE-2025-65858	CWE-79	Calibre-Web	0.6.21	Detected	Blocked
16	CVE-2025-65817	CWE-200	Overhang.IO	20.0.2	Detected	Blocked
17	CVE-2025-65672	CWE-639	Classroomio LMS	0.1.13	Detected	Blocked
18	CVE-2025-65345	CWE-22	alexusmai	≤ 3.3.1	Detected	Blocked
19	CVE-2025-64513	CWE-287	Milvus	2.6.5	Detected	Blocked
20	CVE-2025-64500	CWE-284	Symfony	2.0.0	Detected	Blocked
21	CVE-2025-64495	CWE-79	Open WebUI	≤ 0.6.34	Detected	Blocked
22	CVE-2025-64484	CWE-287	OAuth2-Proxy	< 7.13.0	Detected	Blocked
23	CVE-2025-64459	CWE-89	Django	5.1	Detected	Blocked
24	CVE-2025-64458	CWE-400	Django	5.1	Detected	Blocked
25	CVE-2025-64446	CWE-22	Fortinet FortiWeb	8.0.0	Detected	Blocked
26	CVE-2025-63830	CWE-79	CKFinder	1.4.3	Detected	Blocked
27	CVE-2025-64095	CWE-434	DotNetNuke	< 10.1.1	Detected	Blocked
28	CVE-2025-64027	CWE-79	Snipe-IT	8.3.4	Detected	Blocked
29	CVE-2025-63914	CWE-409	Cinnamon kotaemon	0.11.0	Detected	Blocked
30	CVE-2025-63888	CWE-94	ThinkPHP	5.0.24	Detected	Blocked

(MITM) interception, brute-force attacks, and social engineering attacks. The experimental results are presented in Table 10.

In the case of social engineering attacks, where attackers exploit tactics to steal users' host machines, accounts, and passwords, they can then use the compromised machines to access the protected system, leading to successful attacks.

Overall, the attack simulation results validate the robustness and high detection accuracy of TLBAC against diverse and high-impact vulnerability exploits. Because TLBAC does not rely on specific software components or application semantics, and instead performs association analysis based solely on trusted labels and behavioral features, it offers generalized protection across heterogeneous system environments. These findings substantiate the practical security enhancement provided by TLBAC in real-world high-threat scenarios and highlight its strong potential for deployment in production systems.

Table 10 Experimental results of common attack methods

Attack type	Before protection	After protection
DDoS	Pass	Block
Phishing	Pass	Block
MITM	Pass	Block
Brute-force	Pass	Block
Social engineering	Pass	Pass

Performance scalability and policy isolation in multi-tenant deployment scenarios(RQ4)

To evaluate the performance of the proposed cloud–edge zero-trust access control system in multi-tenant

scenarios, this study constructed an experimental environment based on WireGuard, created 20 mutually isolated virtual tenants on a single Linux host, and deployed an edge decision gateway on the server side to perform real-time identity verification and policy matching for inbound connections, thereby simulating the access control process under concurrent multi-tenant access.

As shown in Table 11, the experiment first examines the system from the perspective of scalability. By setting the number of concurrent tenants to 2, 5, 10, and 20, respectively, and using TCP handshake establishment time as an approximate indicator of policy-processing overhead, the response performance of the system under different load conditions was measured. The results show that when the number of concurrent tenants ranges from 2 to 10, the average system latency remains within 8.2–9.2 ms, indicating that the edge gateway can maintain relatively stable processing performance under a moderate level of tenant concurrency. In contrast, when the number of concurrent tenants increases to 20, system performance degrades significantly: among 300 requests, 150 experience timeouts, corresponding to a timeout rate of 50%. Considering that no substantial increase in server-side CPU or memory utilization was observed during the experiment, it can be inferred that this performance degradation is unlikely to be caused by the exhaustion of computational resources, but is more likely attributable to the capacity limitation of the WireGuard inbound packet queue or the runtime scheduling mechanism. These findings suggest that the stable operating range of the current system implementation lies between 10 and 20 concurrent tenants, and that further optimization of packet-processing capability at the access layer is required to support larger-scale deployments.

From the perspective of inter-tenant performance isolation, a Noisy-Neighbor experiment was conducted. In this experiment, a single tenant (test1) was selected to continuously generate high-concurrency flooding traffic, while the access latency of other normal tenants (test2–test6) was monitored simultaneously. As shown in Fig. 13, in the presence of flooding traffic from a single tenant, the access latency of all normal tenants increased to varying degrees. Specifically, the average latency rose from approximately 7.2 ms under the baseline condition to approximately 8.7 ms during the flooding period, representing an overall increase of about 21%. For individual tenants, the latency of test2 increased from 7.2 ms to 9.0 ms (+25%), test3 from 6.8 ms to 8.4 ms (+23%), test4 from 7.4 ms to 8.0 ms (+8%), test5 from 7.1 ms to 8.0 ms (+12%), and test6 from 7.4 ms to 9.8 ms, showing the largest increase of 32%.

Meanwhile, as illustrated in Fig. 14, the server CPU utilization rapidly increased from an almost idle state to a peak of 26.4%, remaining at a relatively high level throughout the flooding period, whereas the memory RSS remained largely stable. These results indicate that high-intensity traffic generated by a single tenant can introduce a certain degree of performance interference to other tenants, mainly manifested as increased access latency and greater tail-latency variation. However, all requests from the normal tenants were completed successfully, with no connection failures or timeout events observed. This suggests that although the system exhibits a certain degree of noisy-neighbor effect, its impact does not compromise overall service availability, and the system still maintains a basic level of performance isolation under multi-tenant conditions.

As shown in Fig. 15, the system is further validated from the perspective of policy isolation correctness. In the experiment, the 20 tenants were divided into two policy groups: tenants in Group G1 were permitted

Table 11 Multi-tenant concurrency scalability experiment

Tenants	Valid samples	Timeouts	Average latency (ms)
2	30	0	8.7
5	75	0	8.2
10	150	0	9.2
20	150	150	11.7

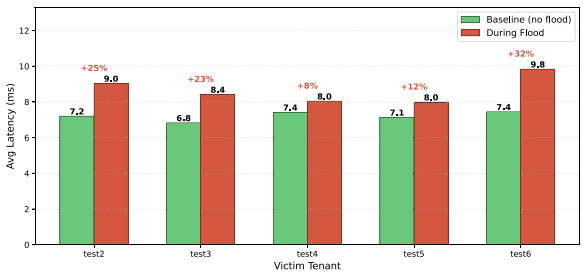


Fig. 13 Average latency comparison of victim tenants under baseline and flood conditions.

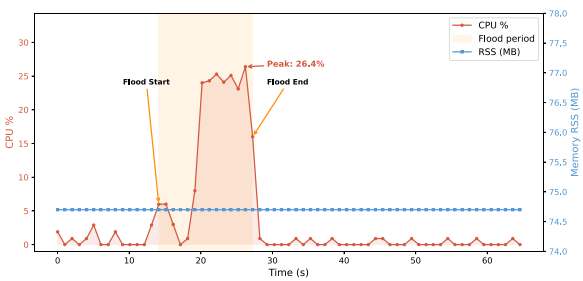


Fig. 14 Temporal variation of server CPU and memory usage under noisy-neighbor flooding.

to access only port 8000, whereas tenants in Group G2 were permitted to access only port 8001. Over 10 rounds of concurrent testing, all tenants simultaneously issued access requests to both ports, resulting in a total of 400 test samples. The experimental results show that the system made correct decisions for all samples, achieving an overall accuracy of 100%, with no instances of cross-tenant unauthorized access observed. These findings indicate that the system can accurately perform tenant identity recognition and access policy matching based on the device token embedded in TCP options together with out-of-band authentication information. As a result, strict policy boundaries are preserved under concurrent conditions, thereby validating the effectiveness and reliability of the proposed multi-tenant access control mechanism.

Overall, the advantages of TLBAC lie in its ability to achieve connection-oriented fine-grained access control with low latency, while simultaneously ensuring policy accuracy and isolation in multi-tenant scenarios. The experiments further show that, under larger-scale concurrency, the primary performance bottlenecks stem from the underlying access and scheduling mechanisms rather than from the policy-processing overhead introduced by TLBAC itself. These findings indicate that TLBAC has strong engineering feasibility and deployment potential, while also providing a clear direction for subsequent performance optimization in large-scale multi-tenant environments.

Discussion

Security considerations of the trusted labels mechanism

The security of TLBAC does not rely on treating Trusted Labels as a cryptographically unforgeable token. Instead, the current design should be understood as a lightweight, context-carrying mechanism for edge-side access control within a cloud–edge–endpoint architecture. In TLBAC, the endpoint generates Trusted Labels using cloud-issued identity credentials together with behavioral features and device-environment attributes collected locally, and embeds them into TCP options for subsequent edge-side inspection. The edge gateway then extracts the labels, correlates them with the corresponding communication context, and evaluates each access request against cloud-distributed policies. Therefore, the security of the mechanism is derived from multi-dimensional consistency checking rather than from the integrity of a single field alone.More specifically, a connection is permitted only when the Trusted Labels simultaneously satisfy the identity, behavior, and device constraints specified by the access control policy. The identity component is a 16-byte credential issued and revoked by the cloud, while the supplementary information includes process-level behavioral features and device fingerprinting attributes collected at the endpoint. As a result, even if an adversary attempts to forge part of the label contents, successful impersonation still requires producing a mutually consistent combination of cloud-issued identity information, runtime behavioral properties, and device context that matches an active policy rule at the edge. This multi-dimensional binding substantially raises the difficulty of unauthorized impersonation compared with schemes that rely solely on static identifiers or network addresses. In addition, the edge layer introduces contextual association and fail-safe enforcement to reduce the risk of label misuse. TLBAC separates communication into a label-carrying packet and a normal authentication packet, and the traffic label identification module associates them through checksum-based correlation. Unmatched packets are buffered only within a bounded waiting window N, after which they are discarded. This design limits mismatching and resource exhaustion under out-of-order delivery or abnormal traffic. Furthermore, when the traffic control decision module cannot retrieve a valid Trusted Label for an incoming SYN request, or when no policy rule is satisfied, the gateway immediately sends a TCP RST and blocks the connection according to a default-deny principle consistent with Zero Trust. Thus, missing, inconsistent, or unrecognized labels do not result in permissive behavior.

Nevertheless, we acknowledge an important limitation of the current design: Trusted Labels, as presently defined, do not provide explicit cryptographic integrity protection or origin authentication. Consequently, under

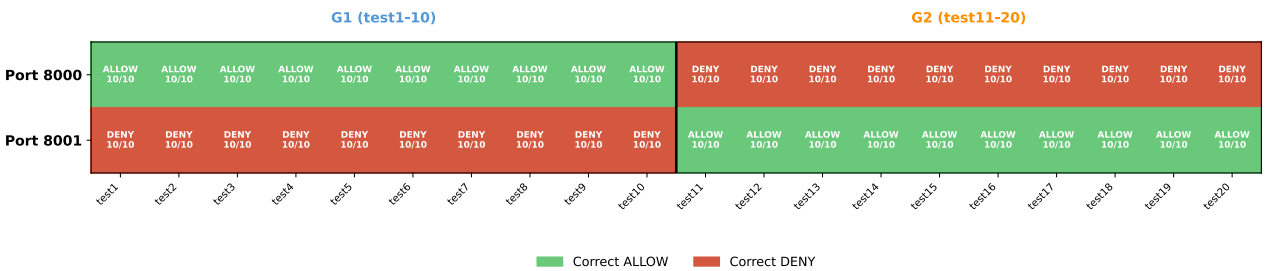


Fig. 15 Multi-tenant policy isolation correctness experiment

a strong adversary model—for example, an attacker capable of modifying TCP options in transit or one who has already compromised a legitimate endpoint—the current mechanism alone cannot guarantee resistance to all tampering or impersonation attempts. In this sense, the present TLBAC design is more accurately characterized as improving the reliability of context-bound authorization and increasing the attack cost for unauthorized access, rather than as fully eliminating forgery at the cryptographic level. This threat boundary is also consistent with the intended deployment model of TLBAC. The mechanism is most effective when deployed in environments where endpoint behavior can be observed, cloud-issued credentials can be managed dynamically, and transport headers are preserved sufficiently for edge enforcement. Within this model, TLBAC provides practical security value by combining cloud-issued identity, endpoint-observed runtime context, checksum-based packet association, and edge-side default-deny policy evaluation into a lightweight Zero Trust access control workflow. As future work, explicit cryptographic protection can be incorporated into Trusted Labels to strengthen resistance against tampering and replay. Possible extensions include attaching an HMAC over label contents, introducing device-bound signatures, or using short-lived nonce- or timestamp-bound tokens issued by the cloud. These enhancements would provide stronger integrity and provenance guarantees while preserving the lightweight edge-side decision model of TLBAC.

Behavioral feature selection and evasion resistance

The behavioral features adopted in TLBAC are not intended to serve as individually evasion-proof indicators; rather, they are selected to improve the robustness of access control decisions through multi-dimensional correlation. As shown in Table 3, the selected features jointly cover process identity (e.g., process identifier, executable path, digital signature, and MD5 hash), execution context (e.g., runtime parameters, parent process, and application window name), and runtime dependency information (e.g., loaded dynamic-link library modules). This design follows three principles: multi-dimensional binding, complementary validation, and increased evasion cost.First, process identity features provide a basic description of the executable that initiates the network request, enabling the system to distinguish among applications at a finer granularity than network addresses or user identities alone. However, static identity-related features may be insufficient when an adversary attempts to mimic legitimate software. For this reason, TLBAC also incorporates execution-context features, such as parent process relationships and runtime parameters, to capture how the process is launched and under what invocation conditions it operates. In addition, runtime dependency information further characterizes the execution environment of the request-generating process, making it more difficult for an attacker to reproduce a fully consistent behavior profile.Second, the selected features are designed to be complementary rather than redundant. A single feature may be spoofed or manipulated in isolation, but simultaneously forging multiple heterogeneous yet correlated attributes is substantially more difficult in practice. For example, imitating a legitimate executable path alone may not be sufficient if the digital signature, process hash, parent process chain, runtime arguments, and loaded libraries do not remain mutually consistent. Therefore, TLBAC does not rely on any single feature as a standalone trust anchor; instead, it improves policy matching reliability by combining multiple partially independent sources of behavioral evidence.At the same time, this design should not be interpreted as providing absolute resistance against advanced evasion. Under a stronger adversary model, especially when the endpoint itself is compromised, an attacker may still be able to manipulate process-level observations or craft malicious behaviors that resemble legitimate execution contexts. Consequently, the practical role of these behavioral features is to raise the cost and complexity of evasion, rather than to eliminate it completely. In this sense, the behavioral feature set in TLBAC should be viewed as a pragmatic mechanism for strengthening context-aware authorization under realistic deployment conditions, while more explicit integrity guarantees and stronger attestation mechanisms remain important directions for future work.

TCP-option embedding and packet-drop risk.

TLBAC carries endpoint-generated trusted labels in the TCP option space (24 bytes) during connection establishment, which can be affected by middleboxes in a few specific situations. In the intended deployment where the edge gateway is placed within a single administrative domain (e.g., enterprise/edge ingress or cloud VPC ingress) and the path avoids TCP-terminating components between the endpoint and the gateway, TCP options are generally preserved similarly to commonly deployed options, so label loss is expected to be uncommon beyond ordinary network loss. Random packet loss is further mitigated by standard TCP retransmissions (e.g., SYN retransmissions), which re-send the option-carrying packets under loss. Label loss may occur when the path includes components such as SYN proxies or TCP-terminating load balancers/proxies that strip, rewrite, or regenerate TCP headers/options.

Conditions under which TLBAC may not achieve optimal performance

Although the evaluation shows that TLBAC introduces low average overhead in the tested environment, its performance may still be affected under certain deployment conditions. In particular, because TLBAC relies on TCP option fields to carry Trusted Labels, middleboxes that rewrite, strip, or normalize TCP options may reduce label recoverability and thus affect enforcement efficiency or lead to conservative flow rejection. In addition, under stronger adversary models such as compromised endpoints or in-path packet modification, stronger verification mechanisms may be required, which could introduce extra computation and latency. Frequent policy updates in highly dynamic cloud environments may also increase synchronization overhead between the cloud and edge layers. These limitations may be mitigated through deployment within a single administrative domain, lightweight integrity protection mechanisms such as HMAC-based authentication or short-lived cloud-issued tokens, and incremental policy dissemination with gateway-side caching to improve stability and robustness.

Edge gateway performance under high-throughput and multi-tenant load

Our evaluation (Table 8) shows low average CPU and memory overhead on a constrained gateway setup, suggesting practical headroom for typical edge deployments. However, high-throughput and multi-tenant settings stress two specific components: (i) *transient state* created by out-of-order arrivals, and (ii) *policy matching* cost when rule sets grow. First, the label identification module may buffer unmatched type-B packets; the bounded waiting window N provides an explicit upper bound on per-flow state lifetime, preventing unbounded memory growth under bursty traffic or adversarial reordering. Second, the connection decision is made at TCP SYN time; the worst-case matching cost grows with the number of active rules (Algorithm 2). This can be reduced without changing the security semantics by: (1) indexing rules by identity (hash lookup) before evaluating behavior and device predicates, (2) short-lived caching of decisions per (tenant, identity, device), and (3) enforcing per-tenant quotas for queue occupancy and connection attempts to avoid noisy-neighbor effects. Under overload or missing labels, the gateway follows a conservative default-deny behavior (RST), preserving the Zero Trust posture.

Real-world deployment constraints: middlebox compatibility and cloud scale

TLBAC embeds trusted labels in TCP options because IP options are frequently sanitized or dropped in operational networks; TCP options are generally more deployable, and we further constrain option size to remain within typical header limits. Nonetheless, some middleboxes (for example, SYN proxies, L4 load balancers, or TCP-terminating proxies) may strip or rewrite TCP options. Therefore, TLBAC is best deployed within a single administrative domain where transport headers are preserved end-to-end (for example, enterprise edge ingress or cloud VPC ingress). If trusted labels cannot be recovered, the gateway safely treats flows as unverified and blocks them, or operators may switch to a detection-only mode for observability when availability is prioritized. For large-scale cloud environments, the Security Cloud Brain can be deployed as a replicated control-plane service that disseminates versioned policies; gateways maintain a local policy cache and apply incremental updates to keep the data plane stable under policy churn. Multi-tenant operation further requires namespaced policies and per-tenant resource limits (for example, bounded queues and rate limits) to ensure isolation and predictable gat.

To evaluate the deployment feasibility of TLBAC in real-world network environments, we conduct a middlebox compatibility experiment to assess whether trusted labels embedded in TCP Options and UDP-based data transmission can be preserved when traffic traverses common network intermediary devices. Experimental Setup.

The experiment is conducted using three physical nodes: (i) an endpoint device (Honor ELP-AN00, Snapdragon 8s Gen 3, Android 16) running the TLBAC client application, (ii) an intermediary server (192.168.9.123) configured with different middlebox software, and (iii) the edge gateway (192.168.9.133) running TLBAC's edge decision module and the backend service. The traffic path is: Endpoint ? Middlebox ? Edge Gateway. Middlebox Configurations. Five middlebox configurations are evaluated: (1) Direct Mode (Baseline). The endpoint communicates directly with the edge gateway without any intermediary device. This configuration serves as the baseline to confirm that trusted labels are correctly generated, transmitted, and verified under normal conditions. (2) iptables NAT (L3). The intermediary server is configured with Linux iptables to perform Network Address Translation. IP forwarding is enabled via sysctl, and DNAT/MASQUERADE rules are applied to forward all incoming TCP and UDP traffic to the edge gateway. Since iptables operates at the network layer (L3), it only modifies IP header fields (source/destination addresses) without inspecting or altering transport-layer headers, thereby preserving TCP Options and UDP payloads intact.(3) HAProxy TCP Mode (L4). HAProxy is deployed on the intermediary server in TCP mode, listening on the business port and forwarding TCP connections to the

edge gateway. In this mode, HAProxy operates as a Layer 4 proxy that relays TCP byte streams without inspecting application-layer content, thus preserving TCP Options including trusted labels. However, HAProxy does not support UDP protocol forwarding, which prevents the transmission of TLBAC's UDP-based data traffic. (4) HAProxy HTTP Mode (L7). HAProxy is configured in HTTP mode on the intermediary server. In this mode, HAProxy terminates the incoming TCP connection from the endpoint, parses the HTTP request at the application layer, and establishes a new TCP connection to the edge gateway. Since the original TCP connection is terminated and a new one is created, all TCP Options from the original connection—including the embedded trusted labels—are discarded. Additionally, HAProxy does not support UDP forwarding in this mode. (5) Nginx Reverse Proxy (L7). Nginx is deployed as an HTTP reverse proxy on the intermediary server. Similar to HAProxy HTTP mode, Nginx terminates the original TCP connection, processes the HTTP request, and initiates a new backend connection. This process strips all original TCP Options, resulting in the loss of trusted labels. UDP traffic is also not forwarded in this configuration.

For each middlebox configuration, the following verification steps are performed. First, the endpoint initiates a standard business access request through the middlebox to the edge gateway. Second, on the edge gateway, tcpdump is used to capture incoming TCP SYN packets and inspect whether the TCP Options field contains the TLBAC trusted labels (Kind = 0x19, 24 bytes). Third, the end-to-end business result is recorded: whether the connection succeeds, whether the edge gateway permits or denies the access based on the presence of valid trusted labels.

The experimental results are summarized in Table 12. The results demonstrate that iptables NAT, operating at L3, fully preserves both TCP Options and UDP payloads, making it fully compatible with TLBAC's trusted label mechanism. HAProxy in TCP mode preserves TCP Options but cannot forward UDP traffic, resulting in incomplete end-to-end communication. Both L7 middleboxes (HAProxy HTTP and Nginx reverse proxy) terminate the original TCP connection and establish new connections to the backend, which strips all original TCP Options including trusted labels. In these cases, the edge gateway correctly identifies the absence of trusted labels and enforces the default-deny policy by sending a TCP RST packet, which is consistent with the Zero Trust principle of failing securely.

These findings confirm that TLBAC is best deployed within network paths where intermediary devices operate at L3 or below, such as enterprise edge ingress points or cloud VPC ingress, where transport-layer headers are preserved end-to-end. When L7 proxies are present in the path, trusted labels will be lost; however, the system degrades safely rather than permissively, as the edge gateway blocks all unverified connections by default.

Future work

While TLBAC demonstrates solid performance in the current TCP-based testing environment, several limitations must be addressed in future research. One key limitation is the system's TCP-only scope. Although TLBAC has proven effective in this context, it is important to explore its scalability and applicability to other protocols, such as UDP and QUIC. These protocols, especially QUIC, are increasingly used in modern internet communication and may introduce new challenges in terms of label propagation and enforcement. Future work will investigate how TLBAC can be extended to support these protocols and address the unique challenges they present.

Another limitation lies in the fact that the current study relies on testing with a single type of Android device. Although this provides a useful testbed for preliminary evaluation, validation across multiple device models remains necessary to ensure the system's robustness in practical deployments.

Additionally, while TLBAC has made strides in fine-grained access control, its resistance to label forgery remains an area of concern. The current system does not provide explicit cryptographic integrity or origin authentication for the Trusted Labels. As future work, we aim to integrate cryptographic protections, such as HMAC-based label authentication or short-lived tokens, to enhance label forgery resistance.

Table 12 Middlebox compatibility evaluation results

Middlebox	Mode	Compatible	Remarks
None (Direct)	N/A	Yes (Baseline)	All trusted labels preserved
iptables NAT	L3	Yes	Does not modify TCP Options or UDP payload
HAProxy TCP	L4	No	TCP Options preserved, but UDP forwarding unsupported
HAProxy HTTP	L7	No	TCP connection terminated, labels stripped, UDP unsupported
Nginx reverse proxy	L7	No	TCP connection terminated, labels stripped, UDP unsupported

Finally, scalability remains a crucial aspect for large-scale deployments. Future extensions could include the integration of TLBAC with **eBPF** for kernel-level enforcement, enabling more granular control and improving system performance. Such an integration would allow TLBAC to operate efficiently in environments with high traffic and complex access control policies, enhancing its applicability in production systems.

In summary, while TLBAC provides a promising solution for fine-grained access control, further research is needed to extend its capabilities to support multiple protocols, improve resistance to label forgery, validate its performance on real devices, and explore scalability through kernel-level enforcement.

Conclusion

This study addresses the practical challenges of fine-grained access control in cloud–edge–endpoint collaborative environments and proposes TLBAC, a trusted-labels–based access control system. By embedding trusted labels into transport-layer TCP packets, TLBAC enables the joint representation and cross-layer propagation of user identity, behavioral attributes, and device context, thereby supporting fine-grained and verifiable access decisions without relying on application-specific logic. The system architecture consists of three layers—cloud-level policy management, edge-level gateway enforcement, and endpoint-level trusted-labels generation—forming a coordinated mechanism that integrates centralized policy orchestration with real-time edge decision-making. In its design, TLBAC constructs trusted labels from three categories of information: identity credentials, behavioral features, and device fingerprints. These labels are securely embedded into TCP packets via the TCP Options field, ensuring both transmission compatibility and trustworthy propagation across heterogeneous protocol environments.

The experimental evaluation is organized around four core research questions. In RQ1, multi-scale TCP transmissions are simulated under high-loss and high-jitter LTE network conditions. The results show that TCP packets carrying trusted labels exhibit no anomalies in segment count, byte consistency, or retransmission behavior, confirming that the embedding process does not compromise protocol reliability. In RQ2, CPU usage, memory consumption, and transmission latency are measured under iptables, detection, direct and protection modes. The findings indicate that even with full protection enabled, the CPU and memory overhead remain extremely low (approximately 0.2%–0.4%), demonstrating suitability for deployment in resource-constrained edge environments. In RQ3, an attack simulation environment containing 30 real-world high-severity vulnerabilities—covering DNS, Log4j2, Spring, Redis, ThinkPHP, and other heterogeneous components—is constructed. Across all attack scenarios, TLBAC successfully detects and blocks malicious traffic without any missed detections or false allowances, validating its robustness and generality in multi-vulnerability, multi-protocol, and multi-threat contexts.In RQ4, the system was evaluated from three aspects: performance scalability, resilience to inter-tenant interference, and the accuracy of policy isolation. The results indicate that the system can maintain low latency under medium-scale multi-tenant concurrent scenarios and continue to provide normal service even in the presence of noisy-neighbor interference. Meanwhile, it achieved 100% policy decision accuracy in multi-tenant concurrent testing, with no cross-tenant unauthorized access observed, thereby demonstrating strong fine-grained access control capability and effective isolation.

Overall, TLBAC provides a trust-based access control mechanism decoupled from traditional network perimeters and grounded in behavioral and identity attributes. It achieves high efficiency, interpretability, and strong defensive capability within complex cloud–edge–endpoint ecosystems. Experimental results demonstrate that TLBAC delivers stable and effective security enhancement under realistic and adversarial threat conditions.

Author contributions

Ru Tan participated in all aspects of this work, contributed the core ideas, and wrote the manuscript. Yingjun Zhang designed the overall framework of the paper. Wei Liu implemented the system described in the paper. Yaqin Cao designed all the experiments. Baihang Liu assisted in conducting the experiments. Xinyu Liu refined and beautified the figures and tables. Qixu Liu and Yuling Liu reviewed the manuscript and provided suggestions for improvements to the details.

Funding

This project is supported by National Natural Science Foundation of China (Grant No.92467201) and the Youth Innovation Promotion Association CAS (Grant No.Y2023047) and High Innovation Plan (N0.202504841069), the Key Laboratory of Network Assessment Technology at Chinese Academy of Sciences and Beijing Key Laboratory of Network Security and Protection Technology.

Declarations

Competing Interests

The authors declare that they have no competing interests.

Author details

[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China. [2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China. [3]Institute of Software, Chinese Academy of Sciences, Beijing 100190, China.

Received: 16 January 2026 Accepted: 9 June 2026
Published online: 22 June 2026

References

(2019) Gb/t 22239-2019, information security technology—baseline for classified protection of cybersecurity. National Standard of the People's Republic of China

(2020) Gb/t 35273-2020, information security technology—personal information security specification. National Standard of the People's Republic of China

(2023) Gb/t 20986-2023, guidelines for classification and grading of cybersecurity incidents. National Standard of the People's Republic of China

(2024) Gb/t 43696-2024, zero trust—reference architecture. National Standard of the People's Republic of China

Ahmed U, Raza I, Hussain SA (2019) Trust evaluation in cross-cloud federation: survey and requirement analysis. ACM Comput Surv 52(1):1–37. https://doi.org/10.1145/3292499

Aldea CL, Bocu R (2025) Authentication challenges and solutions in microservice architectures. Appl Sci 15(22):12088. https://doi.org/10.3390/app152212088

Ali B, Gregory MA, Li S, Dib OA (2024) Implementing zero trust security with dual fuzzy methodology for trust-aware authentication and task offloading in multi-access edge computing. Comput Netw 241(110):197. https://doi.org/10.1016/j.comnet.2024.110197 (https://www.sciencedirect.com/science/article/pii/S138912862400029X)

Anjum I, Kostecki D, Leba E, Sokal J, Bharambe R, Enck W, Nita-Rotaru C, Reaves B (2022) Removing the reliance on perimeters for security using network views. In: Proceedings of the 27th ACM on symposium on access control models and technologies, association for computing machinery, New York, NY, USA, SACMAT '22, p 151–162, https://doi.org/10.1145/3532105.3535029

Atzori L, Iera A, Morabito G (2010) The internet of things: a survey. Comput Netw 54(15):2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010 (https://www.sciencedirect.com/science/article/pii/S1389128610001568)

Deng S, Zhao H, Huang B, Zhang C, Chen F, Deng Y, Yin J, Dustdar S, Zomaya AY (2024) Cloud-native computing: a survey from the perspective of services. Proc IEEE 112(1):12–46. https://doi.org/10.1109/JPROC.2024.3353855

Di Francesco P, Lago P, Malavolta I (2019) Architecting with microservices: a systematic mapping study. J Syst Softw 150:77–97. https://doi.org/10.1016/j.jss.2019.01.001 (https://www.sciencedirect.com/science/article/pii/S0164121219300019)

Diekmann C, Naab J, Korsten A, Carle G (2019) Agile network access control in the container age. IEEE Trans on Netw and Serv Manag 16(1):41–55. https://doi.org/10.1109/TNSM.2018.2889009

dos Santos DR, Marinho R, Schmitt GR, Westphall CM, Westphall CB (2016) A framework and risk assessment approaches for risk-based access control in the cloud. J Netw Comput Appl 74:86–97. https://doi.org/10.1016/j.jnca.2016.08.013 (https://www.sciencedirect.com/science/article/pii/S1084804516301710)

Ferretti L, Magnanini F, Andreolini M, Colajanni M (2021) Survivable zero trust for cloud computing environments. Comput Secur 110(102):419. https://doi.org/10.1016/j.cose.2021.102419 (https://www.sciencedirect.com/science/article/pii/S0167404821002431)

Gambo ML, Almulhem A (2025) Zero trust architecture: a systematic literature review. J Netw Syst Manag 34: 25. https://api.semanticscholar.org/CorpusID:277066000

Grubbs P, Arun A, Zhang Y, Bonneau J, Walfish M (2022) Zero-Knowledge middleboxes. In: 31st USENIX security symposium (USENIX Security 22), USENIX Association, Boston, MA, pp 4255–4272, https://www.usenix.org/conference/usenixsecurity22/presentation/grubbs

Hsieh K, Wong M, Segarra S, Mani SK, Eberl T, Panasyuk A, Netravali R, Chandra R, Kandula S (2024) NetVigil: Robust and Low-Cost anomaly detection for East-West data center security. In: 21st USENIX symposium on networked systems design and implementation (NSDI 24), USENIX Association, Santa Clara, CA, pp 1771–1789, https://www.usenix.org/conference/nsdi24/presentation/hsieh

Lee YT, Enck W, Chen H, Vijayakumar H, Li N, Qian Z, Wang D, Petracca G, Jaeger T (2021) PolyScope: Multi-Policy access control analysis to compute authorized attack operations in android systems. In: 30th USENIX security symposium (USENIX Security 21), USENIX Association, pp 2579–2596, https://www.usenix.org/conference/usenixsecurity21/presentation/lee-yu-tsung

Li X, Chen Y, Lin Z, Wang X, Chen JH (2021) Automatic policy generation for Inter-Service access control of microservices. In: 30th USENIX security symposium (USENIX Security 21), USENIX Association, pp 3971–3988, https://www.usenix.org/conference/usenixsecurity21/presentation/li-xing

Lin L, Mao X, Chu Z, Xie X (2024) Adaptive access control oriented to data life cycle in hybrid cloud environment. J Softw 35(3):1357–1376 ((**in Chinese**))

Liu Y, Xing X, Tong Z, Lin X, Chen J, Guan Z, Wu Q, Susilo W (2024) Secure and scalable cross-domain data sharing in zero-trust cloud-edge-end environment based on sharding blockchain. IEEE Trans Dependable Secure Comput 21(4):2603–2618. https://doi.org/10.1109/TDSC.2023.3313799

Mani SK, Hsieh K, Segarra S, Chandra R, Zhou Y, Kandula S (2025) Securing public cloud networks with efficient role-based micro-segmentation. In: 22nd USENIX symposium on networked systems design and implementation (NSDI 25), USENIX Association, Philadelphia, PA, pp 1033–1048, https://www.usenix.org/conference/nsdi25/presentation/mani

Mao Y, You C, Zhang J, Huang K, Letaief KB (2017) A survey on mobile edge computing: the communication perspective. IEEE Commun Surv Tutor 19(4):2322–2358. https://doi.org/10.1109/COMST.2017.2745201

Moubayed A, Refaey A, Shami A (2019) Software-defined perimeter (SDP): state of the art secure solution for modern networks. IEEE Netw 33(5):226–233. https://doi.org/10.1109/MNET.2019.1800324

Oluwatimi O, Damiani ML, Bertino E (2018) A context-aware system to secure enterprise content: incorporating reliability specifiers. Comput Secur 77:162–178. https://doi.org/10.1016/j.cose.2018.04.001 (https://www.sciencedirect.com/science/article/pii/S0167404818303018)

Oussous A, Benjelloun FZ, Ait Lahcen A, Belfkih S (2018) Big data technologies: a survey. J King Saud Univ Comput Inf Sci 30(4):431–448. https://doi.org/10.1016/j.jksuci.2017.06.001 (https://www.sciencedirect.com/science/article/pii/S1319157817300034)

Pauley E, Domico K, Hoak B, Sheatsley R, Burke Q, Beugin Y, Kirda E, Mcdaniel P (2022) Secure ip address allocation at cloud scale. In: Proceedings 2025 network and distributed system security symposium https://api.semanticscholar.org/CorpusID:253157649

Qi S, Zheng Y (2021) Crypt-DAC: cryptographically enforced dynamic access control in the cloud. IEEE Trans Dependable Secure Comput 18(2):765–779. https://doi.org/10.1109/TDSC.2019.2908164

Raffa G, Blasco J, O'Keeffe D, Dash SK (2025) Cloudflow: identifying security-sensitive data flows in serverless applications. In: Proceedings of the 34th USENIX conference on security symposium, USENIX Association, USA, SEC '25

Rossi M, Facchinetti D, Bacis E, Rosa M, Paraboschi S (2021) SEApp: Bringing mandatory access control to android apps. In: 30th USENIX security symposium (USENIX Security 21), USENIX Association, pp 3613–3630, https://www.usenix.org/conference/usenixsecurity21/presentation/rossi

Sänger N, Abeck S (2024) Fine-grained authorization in microservice architecture: A decentralized approach. In: Proceedings of the 39th ACM/SIGAPP symposium on applied computing, association for computing machinery, New York, NY, USA, SAC '24, p 1219–1222, https://doi.org/10.1145/3605098.3636121

Shang S, Wang X, Liu A (2024) ABAC policy mining method based on hierarchical clustering and relationship extraction. Comput Secur 139(103):717. https://doi.org/10.1016/j.cose.2024.103717 (https://www.sciencedirect.com/science/article/pii/S016740482400018X)

T N N, Pramod D, Singh R (2023) Zero trust security model: Defining new boundaries to organizational network. In: Proceedings of the 2023 fifteenth international conference on contemporary computing, association for computing machinery, New York, NY, USA, IC3-2023, p 603–609. https://doi.org/10.1145/3607947.3608067

Wang Q, Du X (2022) Design of logistics cloud platform based on blockchain. In: Proceedings of the 2022 4th blockchain and internet of things conference, association for computing machinery, New York, NY, USA, BIOTC '22, p 100–106, https://doi.org/10.1145/3559795.3559809

Xu S, Yang G, Mu Y, Deng RH (2018) Secure fine-grained access control and data sharing for dynamic groups in the cloud. IEEE Trans Inf Forensics Secur 13(8):2101–2113. https://doi.org/10.1109/TIFS.2018.2810065

Xue K, Gai N, Hong J, Wei DS, Hong P, Yu N (2022) Efficient and secure attribute-based access control with identical sub-policies frequently used in cloud storage. IEEE Trans Dependable Secure Comput 19(1):635–646. https://doi.org/10.1109/TDSC.2020.2987903

Yang N, Shen W, Li J, Liu X, Guo X, Ma J (2023) Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In: Proceedings of the 2023 ACM SIGSAC conference on computer and communications security, association for computing machinery, New York, NY, USA, CCS '23, p 3048–3062, https://doi.org/10.1145/3576915.3623121,

Yang N, Liu X, Shen W, Li J, Lu K (2025) Dangers behind access control: Understanding and exploiting implicit permissions in kubernetes. In: Proceedings of the 2025 ACM SIGSAC conference on computer and communications security, association for computing machinery, New York, NY, USA, CCS '25, p 3401–3415, https://doi.org/10.1145/3719027.3765106,

Yu S, Wang C, Ren K, Lou W (2010) Achieving secure, scalable, and fine-grained data access control in cloud computing. In: 2010 Proceedings IEEE INFOCOM, pp 1–9. https://doi.org/10.1109/INFCOM.2010.5462174

Zahoor E, Chaudhary M, Akhtar S, Perrin O (2023) A formal approach for the identification of redundant authorization policies in Kubernetes. Comput Secur 135(103):473. https://doi.org/10.1016/j.cose.2023.103473 (https://www.sciencedirect.com/science/article/pii/S0167404823003838)

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.